

UNIVERSITY OF TRANSPORT AND COMMUNICATIONS



GRADUATION THESIS

DEVELOPMENT OF A LICENSE PLATE RECOGNITION SYSTEM API

Student: Trần Hữu Tâm – 222631136

Class: CNCNTTVA – K63

Instructor: TS. Vũ Huấn

Hanoi, May 2026

UNIVERSITY OF TRANSPORT AND COMMUNICATIONS



GRADUATION THESIS

DEVELOPMENT OF A LICENSE PLATE RECOGNITION SYSTEM API

Student: Trần Hữu Tâm – 222631136

Class: CNCNTTVA – K63

Instructor: TS. Vũ Huấn

Hanoi, May 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến các Thầy/Cô trong Ban Giám hiệu, Khoa Công nghệ Thông tin của Trường Đại học Giao thông Vận tải đã tạo điều kiện học tập, cung cấp kiến thức nền tảng vững chắc cho em trong suốt những năm tháng ngồi trên giảng đường.

Đặc biệt, em xin gửi lời tri ân sâu sắc đến TS. Vũ Huấn, người đã trực tiếp hướng dẫn, định hướng và tận tình chỉ bảo, giúp em vượt qua những khó khăn trong suốt quá trình thực hiện Đồ án tốt nghiệp này.

Mặc dù đã nỗ lực tìm hiểu, nghiên cứu và vận dụng các kiến thức đã học, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, đồ án chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý và chỉ bảo quý báu từ Hội đồng bảo vệ và quý Thầy/Cô để hệ thống được hoàn thiện hơn, cũng như giúp em tích lũy thêm kinh nghiệm cho con đường phát triển sự nghiệp sau này.

Em xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Trong kỷ nguyên số hóa hiện nay, sự bùng nổ của Trí tuệ Nhân tạo (AI) và Thị giác máy tính đã mang đến những thay đổi mang tính cách mạng cho mọi lĩnh vực. Đặc biệt, trong bài toán quản lý bãi đỗ xe tự động (Smart Parking), việc nhận diện biển số xe tự động chính là một trong những mắt xích quan trọng nhất để tối ưu hóa quản lý và vận hành.

Thúc đẩy bởi khát vọng "làm chủ công nghệ" và niềm đam mê nghiên cứu đối với Thị giác máy tính, em đã quyết định thực hiện đề tài: **"Nghiên cứu và phát triển API nhận diện biển số xe tự động ứng dụng Deep Learning"**. Thay vì chỉ tập trung xây dựng các ứng dụng quản lý bề nổi, đề án này hướng tới việc tự nghiên cứu, phát triển và làm chủ "phần lõi" thuật toán nhận diện.

Báo cáo này không chỉ ghi lại quá trình nghiên cứu, huấn luyện và triển khai một hệ thống AI thực tiễn, mà còn là minh chứng cho nỗ lực giải quyết bài toán kỹ thuật từ gốc rễ trong suốt thời gian học tập và rèn luyện tại trường.

MỤC LỤC

LỜI CẢM ƠN	3
LỜI NÓI ĐẦU.....	4
DANH MỤC HÌNH ẢNH	8
DANH MỤC BẢNG BIỂU	9
PHẦN MỞ ĐẦU	10
1. Lý do chọn đề tài	10
2. Mục tiêu nghiên cứu	10
3. Đối tượng và phạm vi nghiên cứu	11
4. Phương pháp nghiên cứu.....	11
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT.....	12
1.1. Tổng quan bài toán nhận diện biển số xe (ALPR).....	12
1.1.1. Các thách thức trong thực tế.....	12
1.1.2. Đánh giá một số giải pháp hiện có.....	12
1.2. Cơ sở lý thuyết mạng Học sâu.....	13
1.2.1. Tổng quan về bài toán Phát hiện đối tượng (Object Detection)	13
1.2.2. Nguyên lý hoạt động của họ mô hình YOLO.....	14
1.2.3. Kiến trúc lõi của mô hình YOLO11	14
1.2.4. Mở rộng bài toán với YOLO-Keypoint.....	15
1.2.5. Bài toán Nhận dạng ký tự (OCR) bằng mô hình Phát hiện đối tượng.....	15
1.3. Các thuật toán xử lý hậu kỳ.....	15
1.4. Công nghệ tối ưu và triển khai hệ thống.....	17
1.4.1. Bộ công cụ tối ưu hóa phần cứng Intel OpenVINO	17
1.4.2. FastAPI	18
1.4.3. Docker: Container hóa và chuẩn hóa môi trường triển khai	18
CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC HỆ THỐNG ALPR VÀ XÂY DỰNG TẬP DỮ LIỆU	19
2.1. Kiến trúc luồng xử lý tổng thể.....	19
2.2. Xây dựng và chuẩn bị tập dữ liệu.....	21
2.2.1. Nguồn gốc và quy mô dữ liệu.....	21

2.2.2. Công cụ và quy trình gán nhãn	21
2.2.3. Tiền xử lý dữ liệu (Preprocessing)	23
2.2.4. Phân chia tập dữ liệu (Data Splitting)	23
2.3. Thiết kế kiến trúc API Backend	24
2.3.1. API Endpoint & Input.....	24
2.3.2. Cơ chế xử lý bất đồng bộ.....	24
2.3.3. JSON Response	24
CHƯƠNG 3: XÂY DỰNG, TỐI ƯU HÓA VÀ ĐÁNH GIÁ HỆ THỐNG.....	26
3.1. Huấn luyện và Đánh giá mô hình	26
3.1.1. Thiết lập môi trường huấn luyện và kiểm thử	26
3.1.2. Thiết lập tham số và Tăng cường dữ liệu	26
3.1.3. Kết quả đánh giá mô hình.....	27
3.2. Tối ưu hóa hiệu năng suy luận bằng Intel OpenVINO	35
3.2.1. Trích xuất đồ thị tính toán sang chuẩn ONNX	35
3.2.2. Biên dịch sang định dạng trung gian (OpenVINO IR)	36
3.2.3. Tối ưu hóa cấu hình Runtime (Cơ chế Latency).....	36
3.3. Triển khai API Backend và Đóng gói Container	36
3.3.1. Tích hợp mã nguồn và Xây dựng dịch vụ API	37
3.3.2. Đóng gói ứng dụng với công nghệ Docker	38
3.4. Đánh giá hệ thống tổng thể.....	39
3.4.1. Đánh giá định lượng.....	39
3.4.2. Đánh giá định tính.....	40
3.5. Nghiên cứu thực nghiệm so sánh với Kiến trúc lai (YOLO11 + Thư viện PaddleOCR).....	47
3.5.1. Mục tiêu và phương pháp kiểm thử	47
3.5.2. Kết quả so sánh định lượng	48
3.5.3. Biện luận kỹ thuật và Kết luận.....	48
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	49
1. Kết luận	49
2. Những hạn chế của đồ án	49

3. Hướng phát triển trong tương lai	49
TÀI LIỆU THAM KHẢO.....	51

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ luồng xử lý dữ liệu	19
Hình 2.2. Quy trình gán nhãn Bounding Box cho từng ký tự và màu nền biển số.....	22
Hình 2.3. Quy trình gán nhãn Bounding Box logo ô tô	22
Hình 2.4. Gán nhãn tọa độ 4 góc (Keypoints) cho biển số.....	23
Hình 3.1. Biểu đồ hàm mất mát và các chỉ số mAP trong quá trình huấn luyện.....	27
Hình 3.2. Biểu đồ Precision-Recall đánh giá khả năng định vị hộp bao (Box).....	28
Hình 3.3. Biểu đồ Precision-Recall đánh giá khả năng dự đoán tọa độ 4 góc (Pose)...	29
Hình 3.4. Ma trận nhầm lẫn (Confusion Matrix) của mô hình phát hiện biển số.....	30
Hình 3.5. Biểu đồ quá trình huấn luyện của mô hình OCR và Màu sắc	31
Hình 3.6. Biểu đồ Precision-Recall (trái) và F1-Curve (phải) của mô hình OCR.....	31
Hình 3.7. Ma trận nhầm lẫn chi tiết các lớp ký tự và màu nền biển số.....	32
Hình 3.8. Biểu đồ quá trình huấn luyện của mô hình phân loại phương tiện.....	33
Hình 3.9. Biểu đồ Precision-Recall (trái) và F1-Curve (phải) của mô hình Phân loại phương tiện.....	34
Hình 3.10. Ma trận nhầm lẫn của mô hình Phân loại phương tiện.	35
Hình 3.11. Giao diện Swagger UI của hệ thống.....	37
Hình 3.12. Container hóa ứng dụng trên Docker	38
Hình 3.13. Kết quả nhận diện xe máy trong điều kiện lý tưởng	41
Hình 3.14. Kết quả nhận diện ô tô trong điều kiện lý tưởng	41
Hình 3.15. Kết quả nhận diện trong điều kiện bị lóa sáng + xa.....	42
Hình 3.16. Kết quả nhận diện trong điều kiện góc chụp bị che khuất	43
Hình 3.17. Khả năng nhận diện vượt ngưỡng khi ở góc xa.....	43
Hình 3.18. Khả năng nhận diện vượt ngưỡng khi biển bị che khuất	44
Hình 3.19. YOLO-Pose bỏ sót mục tiêu do nhòe hình.....	45
Hình 3.20. YOLO-Pose bỏ sót mục tiêu do biển số bị che khuất	45
Hình 3.21. Trường hợp hệ thống nhận diện bỏ sót.....	46
Hình 3.22. Trường hợp hệ thống nhận diện sai.....	47

DANH MỤC BẢNG BIỂU

Bảng 2.1. Thống kê và đặc tả các tập dữ liệu huấn luyện	21
Bảng 3.1. Kết quả kiểm thử tổng quan của mô hình	39
Bảng 3.2. Bảng đối chiếu hiệu năng thực nghiệm giữa hệ thống lỗi họ YOLO11 và Kiến trúc lai (YOLO11 + PaddleOCR) trên CPU	48

PHẦN MỞ ĐẦU

1. Lý do chọn đề tài

Tại các cơ sở hạ tầng có lưu lượng giao thông lớn như trung tâm thương mại, trường học, và bệnh viện, hệ thống nhận diện biển số xe (ALPR) đóng vai trò cốt lõi trong việc kiểm soát an ninh và vận hành bãi đỗ xe tự động. Tuy nhiên, việc triển khai thực tế tại các khu vực này thường gặp phải nhiều thách thức như: không gian lắp đặt hạn chế dẫn đến góc chụp khó, điều kiện ánh sáng phức tạp, ... gây khó khăn cho một hệ thống nhận diện biển số.

Bên cạnh đó, xét về mặt kỹ thuật, việc xây dựng một hệ thống nhận diện thông nhất có khả năng xử lý nhiều luồng tác vụ phức tạp với độ trễ thấp trên phần cứng phổ thông (CPU) vẫn là một thách thức. Sự phát triển vượt bậc của kiến trúc mạng học sâu YOLO và công cụ tối ưu OpenVINO đã mở ra hướng tiếp cận hoàn toàn mới để giải quyết triệt để bài toán này. Để khắc phục các góc chụp khó, đề án ứng dụng **YOLO11n-Pose** để định vị chính xác 4 góc biển số, hỗ trợ xoay phẳng hình ảnh trước khi đưa vào **YOLO11n** làm mô hình lõi cho tác vụ nhận dạng ký tự (OCR).

Mặc dù các phiên bản mới hơn (như YOLO26) đã ra mắt, nhưng qua quá trình nghiên cứu, phiên bản này chủ yếu được thiết kế tối ưu cho thiết bị biên (Edge Devices). Ngược lại, YOLO11 là phiên bản đã được kiểm chứng về độ ổn định, cung cấp sự cân bằng vượt trội giữa độ chính xác và tốc độ suy luận. Đặc biệt, cấu trúc mạng của YOLO11 cho phép tích hợp hiệu quả với bộ công cụ OpenVINO, đáp ứng hoàn hảo yêu cầu vận hành thời gian thực trên các hệ thống phần cứng không có GPU rời. Đây chính là nền tảng kỹ thuật then chốt để đề án thực hiện mục tiêu xây dựng một giải pháp API nhận diện biển số độc lập, làm chủ hoàn toàn công nghệ lõi và sẵn sàng tích hợp linh hoạt vào mọi hạ tầng bãi đỗ xe thông minh.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát: Nghiên cứu, xây dựng và làm chủ thành công API lõi nhận diện biển số xe tự động, đảm bảo hoạt động ổn định, chính xác và có thể dễ dàng tích hợp vào các hệ thống quản lý bãi đỗ xe.

Mục tiêu cụ thể:

- Thiết kế một luồng xử lý đồng nhất dựa trên nền tảng kiến trúc YOLO11 để giải quyết 02 bài toán cốt lõi: Phát hiện và nắn chỉnh biển số, Nhận diện ký tự (OCR) kết hợp phân loại màu biển. Bên cạnh đó, nghiên cứu tính khả thi của một mô hình Phân loại phương tiện giao thông để phục vụ các bài toán sau này.
- Ứng dụng framework Intel OpenVINO để chuyển đổi và tối ưu hóa mô hình, nhằm đạt tốc độ suy luận (inference speed) cao ngay cả trên môi trường thiết bị vi xử lý trung tâm CPU.

- Đóng gói toàn bộ hệ thống dưới dạng container (Docker) và xuất bản dịch vụ thông qua FastAPI để sẵn sàng triển khai thực tế.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Các kiến trúc mạng Học sâu (Deep Learning) thuộc họ YOLO, trọng tâm là YOLO11, các thuật toán biến đổi hình học trong xử lý ảnh, và bộ công cụ tối ưu hóa hiệu năng mô hình Intel OpenVINO.

Phạm vi nghiên cứu:

- *Về mặt tính năng:* Đề tài tập trung hoàn toàn vào việc xây dựng "phần lõi" thuật toán và hệ thống Backend API. Phạm vi nghiên cứu không bao gồm việc phát triển các nền tảng giao diện người dùng (Frontend/Web/Mobile App).
- *Về mặt dữ liệu:* Hệ thống được huấn luyện, đánh giá và tinh chỉnh chủ yếu trên tập dữ liệu biển số xe cơ giới đường bộ lưu thông tại Việt Nam, với nguồn dữ liệu phần lớn được thu thập trực tiếp từ môi trường thực tế tại các bãi đỗ xe.
- *Về mặt triển khai:* Quá trình thử nghiệm và tối ưu hóa hiệu năng được thực hiện với mục tiêu vận hành tốt trên kiến trúc phần cứng máy tính cá nhân. Môi trường kiểm thử được thực thi trực tiếp trên vi xử lý trung tâm (Intel Core i5-10300H), không sử dụng bộ xử lý đồ họa (GPU) rời.

4. Phương pháp nghiên cứu

Phương pháp nghiên cứu lý thuyết: Thu thập, tổng hợp và phân tích các tài liệu học thuật, tài liệu kỹ thuật liên quan đến lĩnh vực Computer Vision, kiến trúc mạng YOLO, nền tảng OpenVINO và công nghệ Docker.

Phương pháp nghiên cứu thực nghiệm:

- Tiến hành thu thập, tiền xử lý và dán nhãn dữ liệu theo chuẩn của mô hình.
- Thực hiện huấn luyện các mô hình AI.
- Thực hiện các bài kiểm thử để so sánh, đánh giá định lượng về hiệu năng và độ chính xác sau khi áp dụng các kỹ thuật tối ưu hóa.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1. Tổng quan bài toán nhận diện biển số xe (ALPR)

Nhận diện biển số xe tự động là một bài toán kinh điển trong lĩnh vực Thị giác máy tính. Về bản chất, đây là quá trình trích xuất chuỗi ký tự trên biển số phương tiện từ dữ liệu hình ảnh. Quá trình này bao gồm các bước: xác định vùng biển số, xử lý hình học và nhận dạng ký tự quang học. Mặc dù cơ sở lý thuyết đã phát triển từ lâu, việc áp dụng ALPR vào thực tế vẫn phải đối mặt với nhiều rào cản mang tính vật lý và môi trường.

1.1.1. Các thách thức trong thực tế

Trong môi trường triển khai thực tế của các bãi đỗ xe thông minh, hình ảnh thu thập được từ camera giám sát luôn chứa đựng nhiều nhiễu loạn không thể kiểm soát. Các thách thức kỹ thuật lớn nhất bao gồm:

- **Biến dạng không gian do góc chụp:** Do yêu cầu về hạ tầng, camera có thể được lắp đặt ở vị trí cao/xa, tạo ra các góc chéo so với phương tiện. Điều này làm cho hình ảnh biển số bị thu hẹp, nghiêng, hoặc biến dạng thành hình thang thay vì hình chữ nhật chuẩn. Nếu không có cơ chế phát hiện điểm đặc trưng để nắn chỉnh hình học, các mô hình OCR sẽ khó có thể đọc đúng toàn bộ biển số.
- **Điều kiện ánh sáng phức tạp:** Hệ thống camera ngoài trời chịu ảnh hưởng trực tiếp bởi thời tiết và chu kỳ ngày đêm. Biển số xe có thể bị lóa do đèn pha phản chiếu ngược sáng ban đêm, bị đổ bóng râm từ cây cối, hoặc lóa sáng do ánh nắng gắt ban ngày.
- **Che khuất và xuống cấp vật lý:** Biển số có thể bị che khuất một phần do bùn đất, thanh chắn, hoặc bị cong vênh, xước, bong tróc sơn do quá trình sử dụng.

1.1.2. Đánh giá một số giải pháp hiện có

Để giải quyết bài toán ALPR, các nhà nghiên cứu và kỹ sư đã phát triển nhiều phương pháp khác nhau. Có thể phân loại các giải pháp này thành ba nhóm chính, với những ưu nhược điểm riêng:

Giải pháp 1: Phương pháp Xử lý ảnh truyền thống

- *Cách thức:* Sử dụng các thuật toán phân tích hình thái học, phát hiện biên (Sobel, Canny) hoặc tìm kiếm đường thẳng (Hough Transform) để khoanh vùng biển số và nhận dạng ký tự.
- *Đánh giá:* Phương pháp này tiêu tốn ít tài nguyên và tốc độ cực nhanh. Tuy nhiên, nó cực kỳ nhạy cảm với nhiễu loạn môi trường. Chỉ cần ánh sáng thay đổi hoặc góc chụp hơi nghiêng, thuật toán sẽ thất bại. Hiện nay, giải pháp này không còn đủ độ tin cậy cho các hệ thống Smart Parking hiện đại.

Giải pháp 2: Kiến trúc kết hợp (YOLO và Thư viện OCR có sẵn)

- *Cách thức:* Sử dụng mô hình YOLO để phát hiện đối tượng (Object Detection) và cắt vùng biên số, sau đó truyền hình ảnh đã cắt vào một thư viện nhận dạng chữ viết phổ thông (như PaddleOCR, EasyOCR) để đọc kết quả.
- *Đánh giá:* Đây là cách tiếp cận dễ triển khai ban đầu, nhưng khi đưa vào thực tế lại bộc lộ nhiều nhược điểm:
 - Không tối ưu cho biên số: Các thư viện OCR có sẵn được thiết kế để đọc văn bản trên giấy. Khi gặp biên số Việt Nam (đặc biệt là biên số bị mờ, ký tự bị dính bùn đất, có đỉnh ốc che khuất, ...), độ chính xác giảm sút nghiêm trọng.
 - Độ trễ cao và khó đồng bộ: Việc dữ liệu phải luân chuyển qua lại giữa mô hình YOLO và một engine OCR riêng biệt gây ra độ trễ (latency) lớn. Hơn nữa, kiến trúc lai này rất khó để xuất và tối ưu hóa đồng bộ trên cùng một nền tảng biên dịch.

Giải pháp 3: Hệ thống nhận dạng đồng nhất dựa trên họ mô hình YOLO

- *Cách thức:* Sử dụng duy nhất nền tảng kiến trúc YOLO để giải quyết toàn bộ chu trình từ phát hiện vùng biên số đến nhận diện ký tự quang học.
- *Đánh giá:* Đây là hướng tiếp cận đột phá và cân bằng nhất hiện nay:
 - Hiệu năng vượt trội: Kiến trúc YOLO đã được chứng minh thực tiễn về khả năng tối ưu hóa tuyệt vời giữa tốc độ và độ chính xác.
 - Khả năng tối ưu hóa sâu: Việc chuẩn hóa toàn bộ luồng xử lý trên cùng một họ kiến trúc giúp loại bỏ hoàn toàn độ trễ do luân chuyển dữ liệu giữa các framework khác nhau, từ đó mở ra khả năng tối ưu hóa bằng OpenVINO để chạy thời gian thực ngay trên phần cứng CPU thông thường.
 - Hệ sinh thái hỗ trợ mạnh mẽ: Sự phát triển của các nền tảng quản lý dữ liệu (Roboflow) và các bộ thư viện mã nguồn mở chuyên dụng (Ultralytics) đã cung cấp một quy trình chuẩn hóa. Điều này giúp toàn bộ các khâu từ tiền xử lý dữ liệu, gán nhãn, đến huấn luyện và xuất mô hình trở nên đơn giản hơn, dễ dàng hơn.

1.2. Cơ sở lý thuyết mạng Học sâu

1.2.1. Tổng quan về bài toán Phát hiện đối tượng (Object Detection)

Trong lĩnh vực Thị giác máy tính, Phát hiện đối tượng là một bài toán phức tạp hơn nhiều so với Phân loại ảnh. Trong khi bài toán phân loại chỉ trả lời câu hỏi "***Bức ảnh này chứa cái gì?***", thì phát hiện đối tượng phải trả lời đồng thời hai câu hỏi: "***Trong ảnh có những đối tượng nào?***" và "***Chúng nằm ở đâu?***". Kết quả đầu ra của bài toán này thường là một hộp bao quanh đối tượng kèm theo nhãn phân loại và độ tin cậy.

Các mô hình phát hiện đối tượng hiện đại được chia thành hai trường phái chính:

- **Mô hình hai giai đoạn:** Tiêu biểu là R-CNN, Faster R-CNN. Chúng quét qua ảnh để tìm các vùng có khả năng chứa đối tượng, sau đó mới phân loại các vùng này. Dù có độ chính xác cao, kiến trúc này thường quá nặng và chậm để chạy thời gian thực.
- **Mô hình một giai đoạn:** Tiêu biểu là SSD và YOLO. Nhóm này coi việc phát hiện đối tượng là một bài toán hồi quy duy nhất, dự đoán trực tiếp tọa độ hộp bao và xác suất phân loại chỉ trong một lần quét mạng neural.

1.2.2. Nguyên lý hoạt động của họ mô hình YOLO

YOLO (You Only Look Once) là mạng nơ-ron tích chập (CNN) thuộc trường phái một giai đoạn [1]. Thay vì tạo ra hàng ngàn vùng đề xuất (proposals) như Faster R-CNN, YOLO chia bức ảnh đầu vào thành một lưới (grid) kích thước $S \times S$.

Mỗi ô sẽ chịu trách nhiệm phát hiện các đối tượng có tâm rơi vào chính ô đó. Mỗi ô dự đoán một số lượng hộp bao nhất định, đi kèm với độ tin cậy của hộp bao và xác suất của từng lớp (class probabilities). Nhờ việc quét toàn bộ ảnh đúng một lần để suy luận ra toàn bộ kết quả, YOLO đạt được tốc độ xử lý nhanh, lý tưởng cho các bài toán thời gian thực.

1.2.3. Kiến trúc lõi của mô hình YOLO11

Kiến trúc của YOLO11 được chuẩn hóa dựa trên sự kế thừa và tinh chỉnh các khối đặc trưng nhằm tối ưu hóa sự cân bằng giữa độ chính xác và tài nguyên tính toán. Các thành phần cốt lõi bao gồm:

- **Backbone với khối C3k2:** Thay vì sử dụng các khối C2f truyền thống, YOLO11 ứng dụng khối C3k2 (Cross Stage Partial Bottleneck with 2 convolutions). Cải tiến này cho phép mạng nơ-ron trích xuất các đặc trưng ngữ nghĩa sâu hơn trong khi vẫn duy trì được tốc độ xử lý nhanh. Điều này đặc biệt quan trọng trong việc nhận diện các nét ký tự mảnh và phức tạp trên biển số xe.
- **Cơ chế chú ý PSA (Position Sensitivity Attention):** Mô hình được tích hợp các lớp PSA giúp tập trung năng lực tính toán vào các vùng ảnh có độ nhạy thông tin cao. Trong bài toán nhận diện biển số, cơ chế này giúp loại bỏ nhiễu từ bối cảnh xung quanh và tập trung vào các điểm đặc trưng của vật thể.
- **Cơ chế dự đoán Anchor-free:** YOLO11 loại bỏ việc sử dụng các hộp bao định sẵn (Anchor Boxes) vốn đòi hỏi tinh chỉnh thủ công và tốn kém tài nguyên. Bằng cách dự đoán trực tiếp tâm và kích thước đối tượng, mô hình đạt được sự linh hoạt tối đa khi đối mặt với các biển số có tỷ lệ khung hình đa dạng hoặc bị biến dạng do góc nhìn.

Trong phạm vi đề án này, để tối ưu hóa tốc độ trên CPU, phiên bản YOLO11n (Nano) được lựa chọn làm cấu hình tiêu chuẩn.

1.2.4. Mở rộng bài toán với YOLO-Keypoint

Thông thường, một mô hình YOLO tiêu chuẩn chỉ xuất ra 4 giá trị của hộp bao: tọa độ tâm, chiều rộng và chiều cao. Tuy nhiên, trong môi trường bãi đỗ xe thực tế, biển số xe có thể bị biến dạng phối cảnh (ngiên, lệch, ...).

Để giải quyết vấn đề này, đề án sử dụng biến thể **YOLO-Keypoint** (YOLO-Pose). Bên cạnh nhánh dự đoán Bounding Box, phần Head của mô hình được thiết kế thêm một nhánh hồi quy để dự đoán tọa độ của N điểm đặc trưng. Đối với bài toán ALPR trong đề án này, $N=4$, tương ứng với 4 góc của biển số xe (Trên-Trái, Trên-Phải, Dưới-Phải, Dưới-Trái).

Việc mô hình học sâu trực tiếp nội suy ra 4 tọa độ góc này là cơ sở quan trọng nhất để hệ thống thực hiện thuật toán biến đổi hình học ở bước hậu xử lý (hướng tới việc nắn phẳng biển số trước khi đọc chữ), mang lại độ chính xác vượt trội so với các phương pháp cắt ảnh chữ nhật thông thường.

1.2.5. Bài toán Nhận dạng ký tự (OCR) bằng mô hình Phát hiện đối tượng

Nhận dạng ký tự quang học (OCR) là quá trình chuyển đổi hình ảnh chứa văn bản thành các chuỗi ký tự có thể mã hóa bởi máy tính. Bài toán OCR thường được giải quyết bằng các mạng nơ-ron học chuỗi như Convolutional Recurrent Neural Network (CRNN) kết hợp với CTC Loss.

Tuy nhiên, trong kiến trúc đồng nhất của đề án này, bài toán OCR được giải quyết hoàn toàn thông qua phương pháp Phát hiện đối tượng sử dụng mạng YOLO.

Thay vì nhận diện cả một chuỗi ký tự liền mạch, mô hình YOLO-OD được huấn luyện để coi mỗi ký tự chữ cái (A-Z) và chữ số (0-9) là một lớp đối tượng độc lập. Khi đưa hình ảnh biển số đã được nắn phẳng vào mô hình, YOLO sẽ dự đoán ra hàng loạt hộp bao nhỏ bọc lấy từng ký tự trên biển.

Việc ứng dụng YOLO vào bài toán OCR mang lại tốc độ suy luận nhanh và độ chính xác cực cao đối với các ký tự có kích thước lớn và rõ ràng như trên biển số xe. Tuy nhiên, kết quả đầu ra của mạng YOLO lúc này chỉ là một tập hợp các ký tự rời rạc kèm tọa độ, chưa tạo thành một chuỗi có ý nghĩa. Do đó, hệ thống yêu cầu một bước xử lý logic phụ trợ: **Sắp xếp chuỗi ký tự**. Bằng cách dựa vào tọa độ tâm (x, y) của các hộp bao ký tự thu được, thuật toán sẽ tiến hành phân dòng (đối với biển số 2 dòng) và sắp xếp các ký tự theo quy tắc từ trái sang phải, từ trên xuống dưới, để ghép lại thành chuỗi biển số hoàn chỉnh cuối cùng.

1.3. Các thuật toán xử lý hậu kỳ

Bên cạnh kiến trúc mạng học sâu, quá trình nhận dạng biển số xe tự động không thể thiếu các bước xử lý hậu kỳ nhằm tối ưu hóa và chuẩn hóa kết quả đầu ra của mô hình.

Trong đề án này, hai thuật toán cốt lõi được áp dụng là Non-Maximum Suppression (NMS) và Perspective Transform.

1.3.1. Thuật toán Non-Maximum Suppression - NMS

Đặc thù của các mạng phát hiện đối tượng một giai đoạn như YOLO là chúng thường dự đoán ra rất nhiều hộp bao (Bounding Box) xung quanh cùng một đối tượng (ví dụ: một chiếc xe, một biển số, hoặc một ký tự). Nếu không có cơ chế lọc, hệ thống sẽ trả về kết quả trùng lặp, gây sai lệch nghiêm trọng cho luồng xử lý logic phía sau.

Thuật toán NMS được sử dụng để giải quyết triệt để vấn đề này. Nguyên lý hoạt động của NMS dựa trên chỉ số Intersection over Union (IoU) – tỷ lệ giữa diện tích phần giao nhau và tổng diện tích của hai hộp bao.

Các bước thực thi của thuật toán NMS diễn ra như sau:

1. Tập hợp tất cả các hộp bao dự đoán được và sắp xếp chúng theo thứ tự độ tin cậy (Confidence Score) giảm dần.
2. Chọn hộp bao có độ tin cậy cao nhất làm hộp bao gốc (chuẩn) và đưa vào danh sách kết quả đầu ra.
3. Tính toán chỉ số IoU giữa hộp bao gốc và tất cả các hộp bao còn lại trong tập hợp.
4. Loại bỏ tất cả các hộp bao có chỉ số IoU vượt quá một ngưỡng cho trước. Việc IoU cao chứng tỏ các hộp bao này đang dự đoán cùng một đối tượng với hộp bao gốc nhưng với độ tin cậy thấp hơn.
5. Lặp lại quá trình trên với hộp bao có độ tin cậy cao nhất tiếp theo trong số các hộp bao chưa bị loại bỏ, cho đến khi duyệt hết danh sách.

Nhờ NMS Threshold, hệ thống đảm bảo mỗi đối tượng trên ảnh chỉ được giữ lại duy nhất một hộp bao dự đoán chính xác nhất.

1.3.2. Phép biến đổi phối cảnh (Perspective Transform)

Như đã phân tích ở mục 1.2.4, hình ảnh biển số thu được từ camera giám sát thực tế thường bị biến dạng phối cảnh (nghiêng, chéo, xô lệch) do góc độ lắp đặt camera đa dạng. Nếu trực tiếp cắt ảnh theo Bounding Box hình chữ nhật thông thường và đưa vào mô hình OCR, cấu trúc không gian của các ký tự sẽ bị méo mó, làm giảm đột ngột tỷ lệ nhận diện chính xác.

Phép biến đổi phối cảnh (Perspective Transform) là một kỹ thuật toán học trong xử lý ảnh số giúp thay đổi góc nhìn của đối tượng, "nắn phẳng" biển số từ không gian 3D về dạng hình chữ nhật 2D trực diện. Thuật toán này yêu cầu đầu vào là tọa độ của 4 điểm trên mặt phẳng ban đầu và 4 điểm đích tương ứng trên mặt phẳng mới.

Trong kiến trúc của hệ thống, phép biến đổi được thiết lập như sau:

- **Tọa độ gốc:** Chính là 4 điểm đặc trưng (Keypoints) ở 4 góc của biển số (Trên-Trái, Trên-Phải, Dưới-Phải, Dưới-Trái) được trích xuất trực tiếp từ đầu ra của mạng YOLO-Keypoint.
- **Tọa độ đích:** Là 4 góc của một hình chữ nhật lý tưởng. Kích thước của hình chữ nhật này được tính toán động dựa trên chiều dài và chiều rộng tối đa giữa các điểm Keypoints, hoặc được nội suy theo tỷ lệ kích thước chuẩn của biển số xe cơ giới tại Việt Nam.

Từ hai tập hợp điểm (x_i, y_i) và (u_i, v_i) này, hệ thống sử dụng thuật toán để giải hệ phương trình tuyến tính, tìm ra một ma trận biến đổi phối cảnh M kích thước 3x3. Khi nhân ma trận M này với toàn bộ điểm ảnh trong vùng chứa biển số, kết quả thu được là một hình ảnh biển số đã được trải phẳng hoàn toàn. Bước tiền xử lý mang tính quyết định này giúp loại bỏ mọi nhiễu loạn về góc chụp, tạo ra dữ liệu đầu vào hoàn hảo cho mô hình YOLO thực hiện nhận dạng ký tự ở chặng cuối.

1.4. Công nghệ tối ưu và triển khai hệ thống

Một mô hình AI dù có độ chính xác cao đến đâu nhưng nếu đòi hỏi quá nhiều tài nguyên phần cứng hoặc khó tích hợp thì cũng không mang lại giá trị thực tiễn. Do đó, bên cạnh việc xây dựng kiến trúc thuật toán, đồ án đặc biệt chú trọng vào khâu tối ưu hóa suy luận và đóng gói dịch vụ. Khâu này được thực hiện thông qua bộ ba công nghệ cốt lõi: Intel OpenVINO, FastAPI và Docker.

1.4.1. Bộ công cụ tối ưu hóa phần cứng Intel OpenVINO

Thông thường, các mô hình YOLO được huấn luyện và xuất ra dưới định dạng trọng số của framework PyTorch (đuôi .pt). Định dạng này rất phù hợp cho quá trình huấn luyện trên GPU nhưng lại cực kỳ cồng kềnh và chậm chạp khi thực hiện suy luận trên các thiết bị chỉ có vi xử lý trung tâm CPU.

Để giải quyết bài toán hiệu năng, đồ án ứng dụng **Intel OpenVINO** (Open Visual Inference and Neural network Optimization) [5] – một bộ công cụ mã nguồn mở được thiết kế đặc biệt để tối ưu hóa và tăng tốc quá trình suy luận của mạng nơ-ron sâu, đặc biệt là trên các thiết bị phần cứng của Intel (CPU, iGPU). Quá trình tối ưu hóa bằng OpenVINO diễn ra qua hai giai đoạn chính:

- **Chuyển đổi mô hình:** Công cụ này đọc cấu trúc mạng PyTorch nguyên bản, loại bỏ các lớp (layers) dư thừa chỉ dùng cho huấn luyện, và biên dịch mô hình sang định dạng trung gian **IR** bao gồm hai file: .xml (chứa cấu trúc mạng) và .bin (chứa trọng số đã được tối ưu). Đồng thời, quá trình này có thể áp dụng các kỹ thuật lượng tử hóa để giảm độ chính xác của số thực từ FP32 xuống FP16 hoặc INT8, giúp giảm đáng kể dung lượng mô hình.
- **Tăng tốc suy luận:** OpenVINO loại bỏ hoàn toàn sự cồng kềnh của PyTorch framework. Nó tận dụng tối đa các tập lệnh phần cứng cấp thấp của CPU (như

AVX2, AVX-512) để xử lý ma trận tính toán. Nhờ đó, hệ thống gồm 3 mô hình YOLO vẫn có thể chạy mượt mà, đạt độ trễ thấp (low-latency) ngay trên các máy chủ không trang bị GPU.

1.4.2. FastAPI

Để hệ thống ALPR có thể hoạt động độc lập và cho phép các phần mềm quản lý bên thứ ba (Web, Mobile App, Desktop App) dễ dàng kết nối, toàn bộ luồng thuật toán AI được bọc trong một API Backend. Đồ án sử dụng **FastAPI [6]** – một web framework hiện đại, hiệu năng cao được viết bằng Python [3].

Việc lựa chọn FastAPI mang lại ba lợi thế kỹ thuật mang tính quyết định:

- **Hỗ trợ xử lý bất đồng bộ:** Nhờ được xây dựng trên Starlette và hỗ trợ triệt để chuẩn bất đồng bộ (Asynchronous - `async/await`), FastAPI cho phép hệ thống tiếp nhận và xử lý đồng thời hàng chục luồng hình ảnh gửi về từ nhiều camera khác nhau mà không bị nghẽn (blocking).
- **Kiểm tra kiểu dữ liệu tự động:** Tích hợp sẵn thư viện Pydantic giúp hệ thống tự động kiểm tra tính hợp lệ của dữ liệu đầu vào (ví dụ: định dạng file ảnh, kích thước) và định dạng chuẩn hóa kết quả đầu ra (chuỗi JSON chứa biến số, tọa độ, thời gian) một cách nghiêm ngặt.
- **Tự động sinh tài liệu API:** FastAPI tự động tạo ra giao diện tài liệu Swagger UI chuẩn RESTful. Điều này giúp các lập trình viên frontend bên thứ ba có thể trực tiếp gửi ảnh test và nhận kết quả ngay trên trình duyệt mà không cần tốn thời gian viết code tích hợp thử nghiệm.

1.4.3. Docker: Container hóa và chuẩn hóa môi trường triển khai

Một trong những khó khăn lớn nhất khi triển khai các hệ thống Học sâu là sự xung đột thư viện. Một đoạn code có thể chạy tốt trên máy tính của lập trình viên nhưng lại báo lỗi khi đưa lên máy chủ do sai lệch phiên bản hệ điều hành, khác phiên bản Python, hoặc thiếu các thư viện đồ họa C++ của OpenCV.

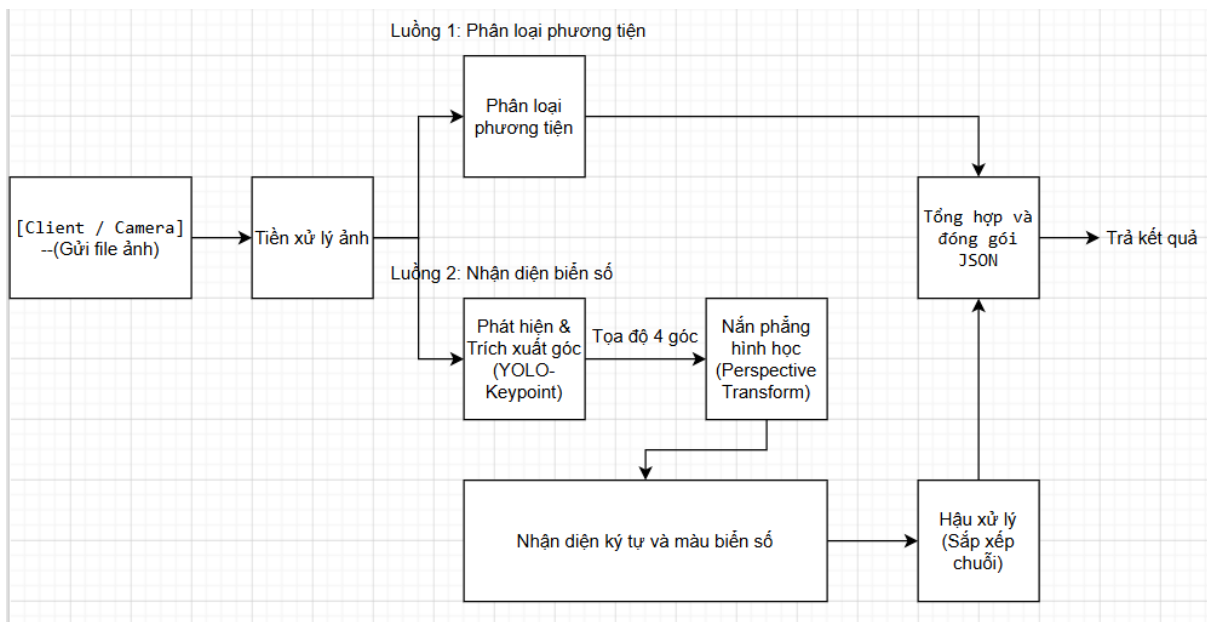
Để triệt tiêu hoàn toàn rủi ro này, hệ thống ALPR của đồ án được đóng gói bằng công nghệ **Docker [7]**. Bản chất của Docker là ảo hóa ở cấp độ hệ điều hành (OS-level virtualization). Toàn bộ mã nguồn YOLO, bộ trọng số OpenVINO, framework FastAPI và mọi thư viện phụ thuộc sẽ được "đóng băng" vào trong một khối độc lập gọi là **Container**.

Nhờ kiến trúc Container này, hệ thống đạt được tính di động tuyệt đối. Chỉ cần một máy chủ hoặc thiết bị biên có cài đặt nền tảng Docker Engine, hệ thống nhận dạng biến số có thể được khởi chạy chỉ với một dòng lệnh duy nhất, đảm bảo môi trường vận hành giống hệt 100% so với lúc lập trình, mở ra khả năng mở rộng hệ thống cực kỳ dễ dàng trong tương lai.

CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC HỆ THỐNG ALPR VÀ XÂY DỰNG TẬP DỮ LIỆU

2.1. Kiến trúc luồng xử lý tổng thể

Để đảm bảo tính đồng bộ, tốc độ xử lý và khả năng mở rộng, hệ thống được thiết kế theo kiến trúc luồng dữ liệu định hướng. Bức ảnh đầu vào sau khi được tiếp nhận sẽ đi qua một chuỗi các trạm xử lý. Thay vì chỉ chạy nối tiếp một đường duy nhất, luồng dữ liệu được phân nhánh thông minh để giải quyết song song các bài toán khác nhau, sau đó hội tụ lại ở bước cuối. Toàn bộ Pipeline này được bọc bên trong một API server.



Hình 2.1. Sơ đồ luồng xử lý dữ liệu

Dựa vào sơ đồ thiết kế tổng thể, quá trình vận hành của hệ thống được chia thành 3 giai đoạn cụ thể như sau:

Bước 1: Tiếp nhận và Tiền xử lý dữ liệu

Hệ thống API tiếp nhận các yêu cầu (HTTP Requests) chứa hình ảnh trực tiếp từ thiết bị Camera giám sát hoặc từ các phần mềm máy khách. Tại thời điểm tiếp nhận, dữ liệu truyền tải trên mạng đang ở dạng luồng byte thô (raw bytes). Hệ thống được thiết kế để lập tức thực hiện bước tiền xử lý: giải mã (decode) luồng dữ liệu này thành ma trận điểm ảnh đa chiều (không gian màu chuẩn) nhằm tạo ra định dạng đầu vào đồng nhất cho toàn bộ các khối AI phía sau.

Bước 2: Xử lý đa luồng bất đồng bộ

Ngay khi bức ảnh được nạp thành công vào lõi hệ thống, bộ điều phối trung tâm sẽ khởi tạo cơ chế xử lý đa luồng bất đồng bộ. Dữ liệu hình ảnh được nhân bản và đẩy đồng thời vào hai luồng tác vụ chạy song song hoàn toàn độc lập, giúp tối ưu hóa tối đa độ trễ (latency) của toàn hệ thống:

- **Luồng 1 - Phân loại phương tiện (Vehicle Classification):** Bức ảnh toàn cảnh được đưa qua một mô hình YOLO chuyên biệt được huấn luyện để nhận diện phương tiện xe máy và các logo/thương hiệu của phương tiện bốn bánh. Tại trạm hậu xử lý của luồng này, một cơ chế ánh xạ logic (mapping) được thiết lập: nếu mô hình phát hiện đặc trưng chung của xe hai bánh, kết quả sẽ được quy định là "xe máy"; nếu phát hiện các đặc trưng thương hiệu (như Toyota, Vinfast, ...), kết quả sẽ được phân lớp và quy định là "ô tô" kèm tên thương hiệu; trong trường hợp không có đối tượng hợp lệ nào vượt qua ngưỡng tin cậy, hệ thống sẽ trả về trạng thái "không xác định" (unknown).
- **Luồng 2 - Nhận diện biển số (ALPR):** Đây là chuỗi xử lý phức tạp nhất, gồm 4 trạm nối tiếp nhau:
 1. *Phát hiện & Trích xuất góc:* Khối YOLO-Keypoint quét toàn ảnh để tìm vị trí hộp bao (Bounding Box) của biển số, đồng thời dự đoán ra tọa độ không gian của 4 điểm góc vật lý của biển.
 2. *Nắn phẳng hình học:* Khối Perspective Transform nhận tọa độ 4 góc, tính toán kích thước động và thực hiện phép biến đổi ma trận để "trải phẳng" vùng ảnh biển số bị biến dạng về hình chữ nhật trực diện. Bức ảnh nhỏ sau khi cắt (crop) được chuyển sang trạm tiếp theo.
 3. *Nhận diện ký tự và màu biển số:* Vùng ảnh biển số đã nắn phẳng được đưa vào mô hình YOLO cuối cùng. Hệ thống thiết kế mô hình này theo hướng đa nhiệm: nó vừa đóng khung dự đoán từng ký tự rời rạc, vừa phân loại màu nền của biển số.
 4. *Hậu xử lý (Sắp xếp chuỗi):* Do các ký tự được AI dự đoán không theo thứ tự không gian tuyến tính, một thuật toán sắp xếp logic (Spatial Sorting) được áp dụng. Dựa vào trục tọa độ y, hệ thống tách biển số thành 1 dòng hoặc 2 dòng, sau đó sắp xếp theo trục x (từ trái qua phải) để ghép các ký tự đơn lẻ thành một chuỗi văn bản hoàn chỉnh.

Bước 3: Tổng hợp và đóng gói kết quả (JSON Packaging)

Đầu ra của Luồng 1 (Loại phương tiện (nếu có), Thương hiệu (nếu có)) và Luồng 2 (Chuỗi ký tự biển số, Màu biển, Độ tin cậy) được trạm cuối của hệ thống tổng hợp lại và tính toán tổng thời gian phản hồi. Nhằm tối ưu hóa tốc độ xử lý của API và tiết kiệm tối đa băng thông mạng, kết quả đầu ra được thiết kế theo nguyên tắc "tinh gọn": chỉ đóng gói dưới định dạng văn bản JSON chuẩn. Hệ thống chủ động loại bỏ quá trình kết xuất ma trận ảnh đồ họa (như vẽ hộp bao lên ảnh gốc hoặc mã hóa base64), giúp API đạt tốc độ phản hồi cực thấp, đáp ứng hoàn hảo tiêu chuẩn của hệ thống quản lý bãi xe thông minh.

2.2. Xây dựng và chuẩn bị tập dữ liệu

Dữ liệu đóng vai trò quyết định đến độ chính xác và khả năng tổng quát hóa của bất kỳ hệ thống Học sâu nào. Do đó, quá trình xây dựng tập dữ liệu (Data Pipeline) được thực hiện vô cùng nghiêm ngặt, trải qua nhiều vòng lặp thu thập - huấn luyện - tinh chỉnh để đạt được bộ trọng số tối ưu nhất.

2.2.1. Nguồn gốc và quy mô dữ liệu

Để đảm bảo tính thực tiễn cao nhất khi đưa hệ thống vào triển khai, dữ liệu được ưu tiên thu thập trực tiếp từ các môi trường thực tế. Cụ thể, sau nhiều giai đoạn tổng hợp và tinh lọc, quy mô bộ dữ liệu cho 3 mô hình được chốt lại như sau:

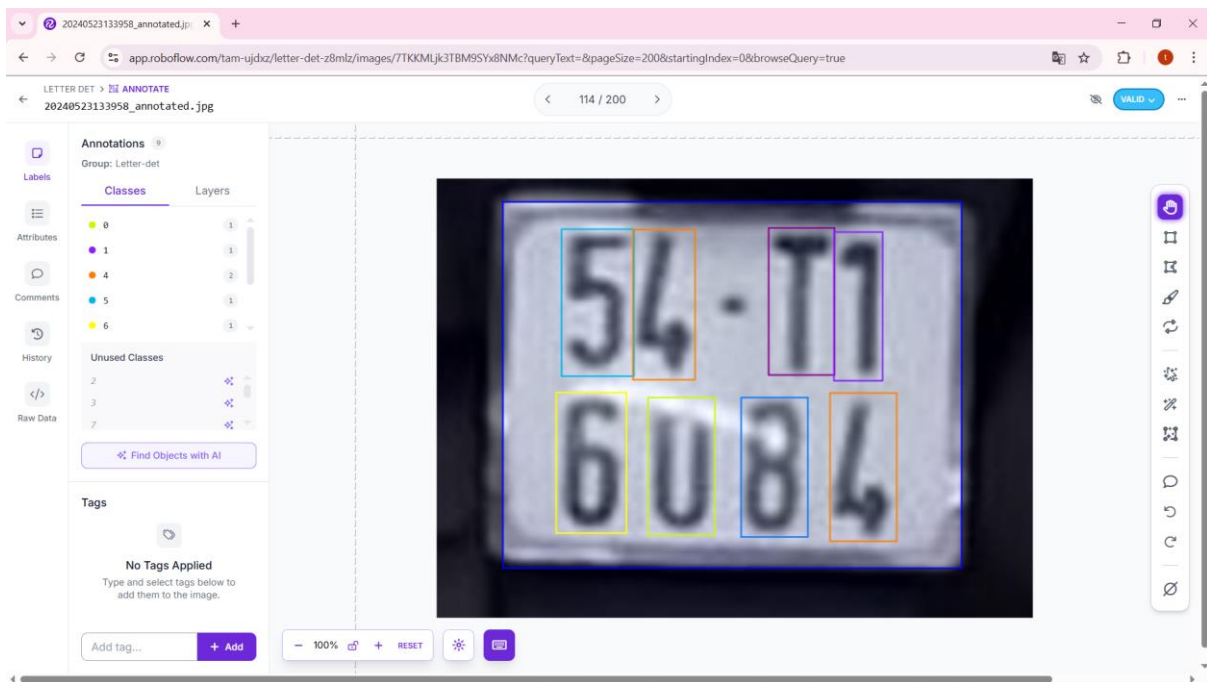
Bảng 2.1. Thống kê và đặc tả các tập dữ liệu huấn luyện

Tập dữ liệu	Số lượng	Nguồn thu thập	Mục đích
Phân loại phương tiện	~ 400 ảnh	Camera giám sát thực tế.	Nhận diện và phân loại ô tô, xe máy.
Phát hiện biển số	~ 1300 ảnh	Camera giám sát thực tế.	Cắt vùng biển và dự đoán tọa độ 4 điểm góc.
Nhận diện ký tự	~ 8700 ảnh	Roboflow & Cắt thủ công (từ dữ liệu camera giám sát thực tế).	Đọc chữ/số và phân loại màu nền biển số.

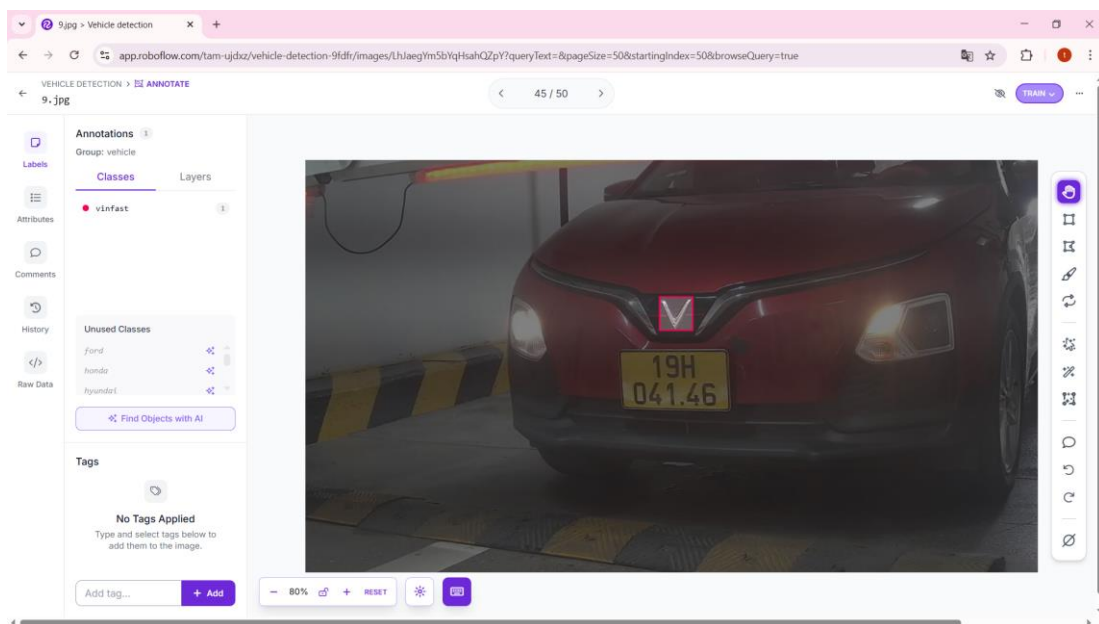
2.2.2. Công cụ và quy trình gán nhãn

Quá trình dán nhãn (Labeling) được thực hiện thủ công và đồng bộ hóa trên nền tảng **Roboflow** [9], giúp quản lý phiên bản và xuất dữ liệu chuẩn định dạng YOLO. Cách thức gán nhãn được tùy biến cho từng loại kiến trúc mô hình:

- **Đối với bài toán Đọc ký tự OCR và phân loại phương tiện:** sử dụng công cụ hộp bao (Bounding Box) tiêu chuẩn. Bounding Box được vẽ bao bọc sát nhất có thể với các cạnh của đối tượng (xe/logo/ký tự/toàn bộ biển số để phân loại màu) nhằm giảm thiểu nhiễu nền (background noise).

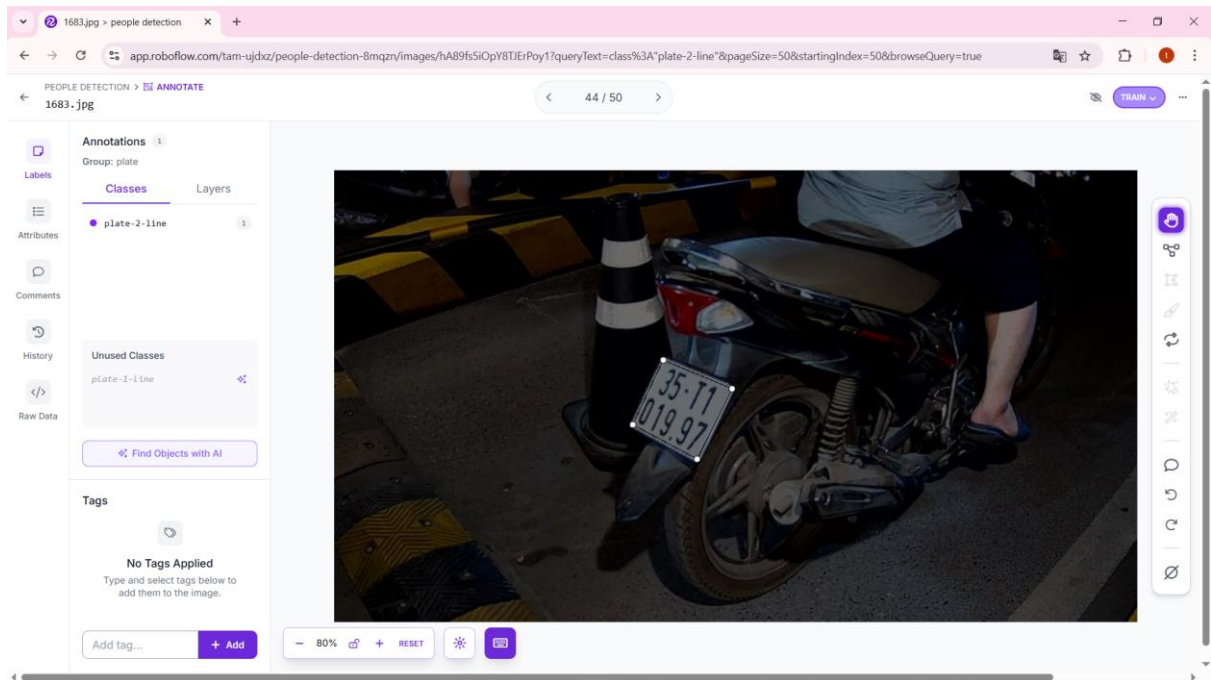


Hình 2.2. Quy trình gán nhãn Bounding Box cho từng ký tự và màu nền biển số



Hình 2.3. Quy trình gán nhãn Bounding Box logo ô tô

- **Đối với bài toán Phát hiện biển số (YOLO-Pose):** Đây là khâu phức tạp nhất. Bên cạnh việc vẽ Bounding Box để khoanh vùng vị trí biển số, hệ thống yêu cầu gán thêm điểm đặc trưng (Keypoints): ở trong Bounding Box khi đó có thêm một hình tứ giác nhỏ và cần phải kéo thả 4 điểm là đỉnh của hình tứ giác đó vào đúng 4 góc của biển số.



Hình 2.4. Gán nhãn tọa độ 4 góc (Keypoints) cho biển số.

2.2.3. Tiền xử lý dữ liệu (Preprocessing)

Trước khi xuất (export) tập dữ liệu để tải về môi trường cục bộ chuẩn bị cho khâu huấn luyện, hình ảnh được cấu hình đi qua các bước tiền xử lý cơ bản trên nền tảng Roboflow [9] nhằm chuẩn hóa định dạng đầu vào:

- **Auto-Orient (Tự động định hướng):** Tự động xoay chiều hình ảnh về đúng hướng không gian chuẩn dựa trên siêu dữ liệu EXIF (Exchangeable Image File Format) của thiết bị chụp. Thao tác này giúp loại bỏ triệt để các trường hợp ảnh bị lật ngược hoặc nằm ngang do lỗi cảm biến của thiết bị thu hình.
- **Resize (Đổi kích thước):** Toàn bộ hình ảnh trong tập dữ liệu, bất kể độ phân giải gốc, đều được thu phóng về kích thước tiêu chuẩn **640x640** pixel. Việc này giúp đồng bộ hóa không gian tensor đầu vào cho mạng YOLO (phục vụ huấn luyện), cân bằng tối ưu giữa việc giữ lại các chi tiết nhỏ (như góc biển, nét chữ) và đáp ứng giới hạn tài nguyên của phần cứng.

2.2.4. Phân chia tập dữ liệu (Data Splitting)

Mỗi tập dữ liệu sau khi chuẩn hóa được phân chia ngẫu nhiên thành 2 tập con để phục vụ cho quy trình huấn luyện chính:

- **Tập huấn luyện (Training Set):** Dùng để trực tiếp cập nhật trọng số của mạng trong quá trình học.
- **Tập xác thực (Validation Set):** Dùng để đánh giá độc lập sau mỗi vòng lặp (epoch), giúp kiểm soát chất lượng học và lưu lại phiên bản trọng số tốt nhất.

Tập kiểm thử (Testing Set): Trong đồ án này, tập kiểm thử không được cắt ra từ dữ liệu huấn luyện. Thay vào đó, đây là một tập dữ liệu riêng biệt, được thu thập thêm hoàn toàn mới từ môi trường thực tế (unseen data) để phục vụ mục đích kiểm thử khách quan độ chính xác của mô hình và định hướng nâng cấp hệ thống trong tương lai.

2.3. Thiết kế kiến trúc API Backend

Đề luồng xử lý AI có thể hoạt động độc lập như một dịch vụ lõi (Core Service) và sẵn sàng tích hợp với các nền tảng quản lý bên thứ ba, hệ thống được bọc bên trong một API xây dựng trên framework FastAPI. Kiến trúc API được thiết kế theo chuẩn RESTful, chú trọng vào khả năng xử lý đồng thời và tối ưu hóa băng thông truyền tải.

2.3.1. API Endpoint & Input

Hệ thống cung cấp một Endpoint duy nhất chịu trách nhiệm tiếp nhận yêu cầu phân tích hình ảnh từ luồng camera giám sát.

- **Đường dẫn (Endpoint):** /predict
- **Phương thức (Method):** POST
- **Định dạng tải lên:** multipart/form-data
- **Tham số (Parameter):** Nhận một biến file chứa dữ liệu hình ảnh nhị phân (ảnh chụp từ camera giám sát). Khối API sẽ tự động đọc luồng byte này vào bộ nhớ (RAM) và giải mã bằng OpenCV [2] mà không cần lưu trữ vật lý xuống ổ cứng.
- **Kiểm soát ngoại lệ:** Hệ thống tích hợp một lớp kiểm tra tính hợp lệ. Nếu dữ liệu truyền vào bị hỏng hoặc không đúng định dạng hình ảnh, API sẽ lập tức chặn luồng xử lý và trả về mã lỗi HTTP 400 (Bad Request), giúp bảo vệ tài nguyên tính toán của máy chủ.

2.3.2. Cơ chế xử lý bất đồng bộ

Dựa trên kiến trúc luồng dữ liệu đa nhánh, API áp dụng kỹ thuật xử lý bất đồng bộ kết hợp với nhóm luồng (Threadpool). Cụ thể, khi một yêu cầu được gửi tới, các tác vụ tính toán nặng của mạng YOLO (như phân loại phương tiện và phát hiện biển số) sẽ được điều phối sang các luồng công việc chạy nền (Background threads). Cơ chế này đảm bảo luồng xử lý chính của API không bị tắc nghẽn (Non-blocking), cho phép hệ thống duy trì khả năng tiếp nhận đồng thời nhiều yêu cầu từ các camera khác nhau trong khi các tác vụ AI đang thực thi.

2.3.3. JSON Response

Nhằm tối ưu hóa tốc độ luân chuyển gói tin trên hạ tầng mạng, hệ thống tuân thủ nguyên tắc trả kết quả dưới dạng văn bản thuần túy (Text-only), loại bỏ việc mã hóa hình ảnh (Base64) trong phản hồi. Cấu trúc JSON được phân tầng để phía máy khách (Client) dễ dàng bóc tách dữ liệu:

- **Thông tin trạng thái:**
 - status: Trạng thái giao tiếp (ví dụ: "success").
 - message: Thông báo trạng thái xử lý logic (ví dụ: "Nhận diện thành công" hoặc "Không tìm thấy biển số").
 - total_time_ms: Tổng thời gian thực tế máy chủ tiêu tốn để xử lý bức ảnh (đơn vị: mili-giây).
- **Khối kết quả (results):** Bao gồm hai mảng dữ liệu song song:
 - **Mảng vehicles (Phương tiện):** Trả về danh sách các phương tiện phát hiện được, bao gồm: loại xe, hãng xe, độ tin cậy và tọa độ khung bao trên ảnh gốc.
 - **Mảng plates (Biển số):** Trả về thông tin chi tiết của các biển số xe trích xuất được, bao gồm: chuỗi ký tự đã ghép hoàn chỉnh (plate_text), màu nền biển số, độ tin cậy của thuật toán OCR và tọa độ vị trí.

Đặc biệt, hệ thống xử lý các ngoại lệ logic tại luồng nhận diện biển số:

- Nếu mô hình phát hiện biển số không tìm thấy đối tượng nào, trường message sẽ thông báo *"Không tìm thấy biển số"*.
- Nếu biển số được tìm thấy nhưng luồng OCR không thể nhận diện được ký tự nào (do ảnh quá mờ hoặc nhiễu), trường plate_text sẽ trả về một chuỗi rỗng (""). Thiết kế này giúp phía Client chủ động thực hiện các kịch bản xử lý logic tiếp theo mà không làm gián đoạn luồng vận hành của ứng dụng quản lý.

CHƯƠNG 3: XÂY DỰNG, TỐI ƯU HÓA VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Huấn luyện và Đánh giá mô hình

Sau khi hoàn thiện khâu chuẩn bị dữ liệu, bước tiếp theo là thiết lập môi trường và cấu hình quá trình học (Training) cho các mạng YOLO. Quá trình này đòi hỏi về tài nguyên phần cứng và việc tinh chỉnh các tham số cùng với các chiến lược tối ưu hóa hậu huấn luyện.

3.1.1. Thiết lập môi trường huấn luyện và kiểm thử

Việc huấn luyện các mạng nơ-ron sâu nhiều tham số đòi hỏi năng lực tính toán cực lớn. Để giải quyết giới hạn về tài nguyên phần cứng cá nhân, toàn bộ quá trình huấn luyện của đồ án được triển khai trên nền tảng điện toán đám mây Google Colaboratory (Colab).

- **Môi trường huấn luyện:** Hệ thống sử dụng máy chủ ảo được cấp phát bộ xử lý đồ họa (GPU) **NVIDIA Tesla T4** với dung lượng VRAM 16GB, kết hợp cùng ~12GB RAM hệ thống. Mức dung lượng VRAM 16GB của T4 cho phép mô hình duy trì kích thước lô (Batch size) ổn định, đáp ứng tốt khả năng tính toán ma trận song song (parallel computing) cho kích thước ảnh đầu vào 640x640 mà không gặp hiện tượng tràn bộ nhớ (Out of Memory - OOM).
- **Môi trường kiểm thử:** Sau khi trích xuất bộ trọng số tốt nhất, quá trình kiểm thử độc lập và chạy API được thực thi cục bộ trên máy tính cá nhân (Asus TUF Gaming fx-506lh). Việc đánh giá trên thiết bị cục bộ nhằm đo lường chính xác độ trễ phản hồi (latency) của hệ thống trong điều kiện tài nguyên giới hạn (chạy trên CPU), qua đó minh chứng tính khả thi của đồ án khi triển khai thực tế.

3.1.2. Thiết lập tham số và Tăng cường dữ liệu

Để mô hình hội tụ tốt và đạt độ chính xác cao trên môi trường Google Colab T4, các siêu tham số cốt lõi được thiết lập theo chiến lược tinh chỉnh linh hoạt (Dynamic Tuning):

- **Model:** khởi tạo bằng model YOLO11n và YOLO11n-pose.
- **Epochs:** Khởi tạo mặc định ở mức 100 epochs cho các chu kỳ huấn luyện lần đầu. Ở những lần huấn luyện tăng cường sẽ giảm số epochs đi, tùy thuộc vào kết quả ở lần huấn luyện gần nhất.
- **Batch Size:** Hệ thống áp dụng chiến lược điều chỉnh động dựa trên bài toán và giai đoạn huấn luyện. Ở giai đoạn đầu, Batch Size được đẩy lên tối đa 64 để tận dụng 16GB VRAM của GPU T4, giúp tăng tốc độ tính toán gradient. Trong các giai đoạn sau, tùy thuộc vào quy mô và độ phức tạp của dữ liệu bổ sung, kích thước lô được chủ động giảm xuống 32 hoặc 16. Việc giảm kích thước lô giúp tăng tính ngẫu nhiên (stochasticity) trong quá trình cập nhật trọng số, hỗ trợ mạng

neuron tăng khả năng tổng quát hóa và "học sâu" hơn vào các đặc trưng vi mô của từng ký tự, biển số.

- **Optimizer:** Giao phó cho cơ chế tự động (Auto) của bộ khung Ultralytics [8]. Dựa trên quy mô tham số của từng biến thể mạng YOLO, framework sẽ tự động phân tích và lựa chọn bộ tối ưu hóa phù hợp nhất (thường là AdamW hoặc SGD), kết hợp cùng lịch trình giảm tốc độ học (Learning rate scheduler) tương ứng.
- **Data Augmentation Constraints:** Hệ thống tận dụng các kỹ thuật tăng cường dữ liệu động của Ultralytics [8] như Mosaic và biến đổi màu sắc HSV. Tuy nhiên, một cấu hình ép buộc được thiết lập: **vô hiệu hóa hoàn toàn** phép lật ngang (Horizontal Flip) và lật dọc (Vertical Flip). Đây là tùy chỉnh bắt buộc đối với bài toán ALPR, bởi việc lật ảnh sẽ làm biến dạng sai lệch cấu trúc không gian của ký tự (ví dụ: số bị lật ngược) và làm sai thực tế của biển số xe.

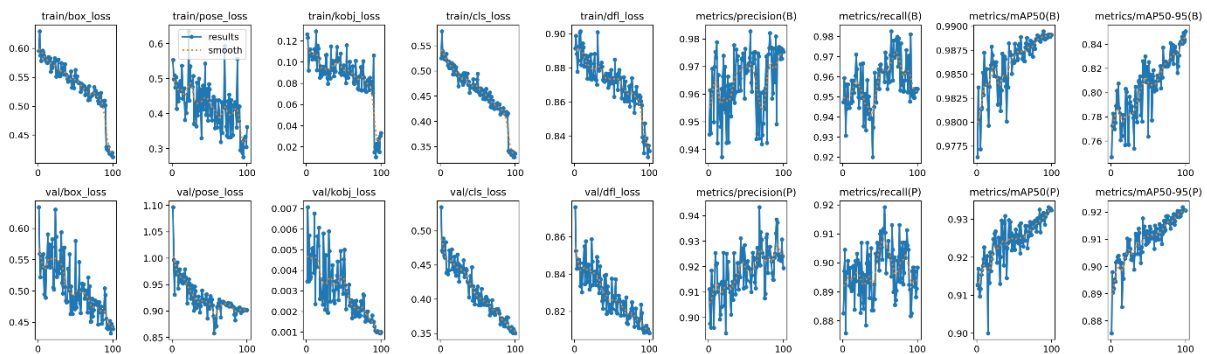
3.1.3. Kết quả đánh giá mô hình

Sau khi quá trình huấn luyện hoàn tất, hiệu năng của các mô hình được đánh giá toàn diện dựa trên tập dữ liệu kiểm thử. Các chỉ số đánh giá cốt lõi được sử dụng bao gồm: Độ chính xác (Precision - P), Độ phủ (Recall - R) và Độ chính xác trung bình (Mean Average Precision - mAP@0.5). Sự hội tụ của các hàm mất mát cũng được phân tích để đảm bảo tính ổn định của mạng neuron.

3.1.3.1. Đánh giá mô hình Phát hiện (YOLO-Pose)

Đây là mô hình quan trọng nhất, đóng vai trò "mở đường" cho thuật toán nắn phẳng phối cảnh phía sau, đảm nhiệm đồng thời hai tác vụ: định vị vùng chứa biển số (Bounding Box) và dự đoán tọa độ 4 góc vật lý (Keypoints).

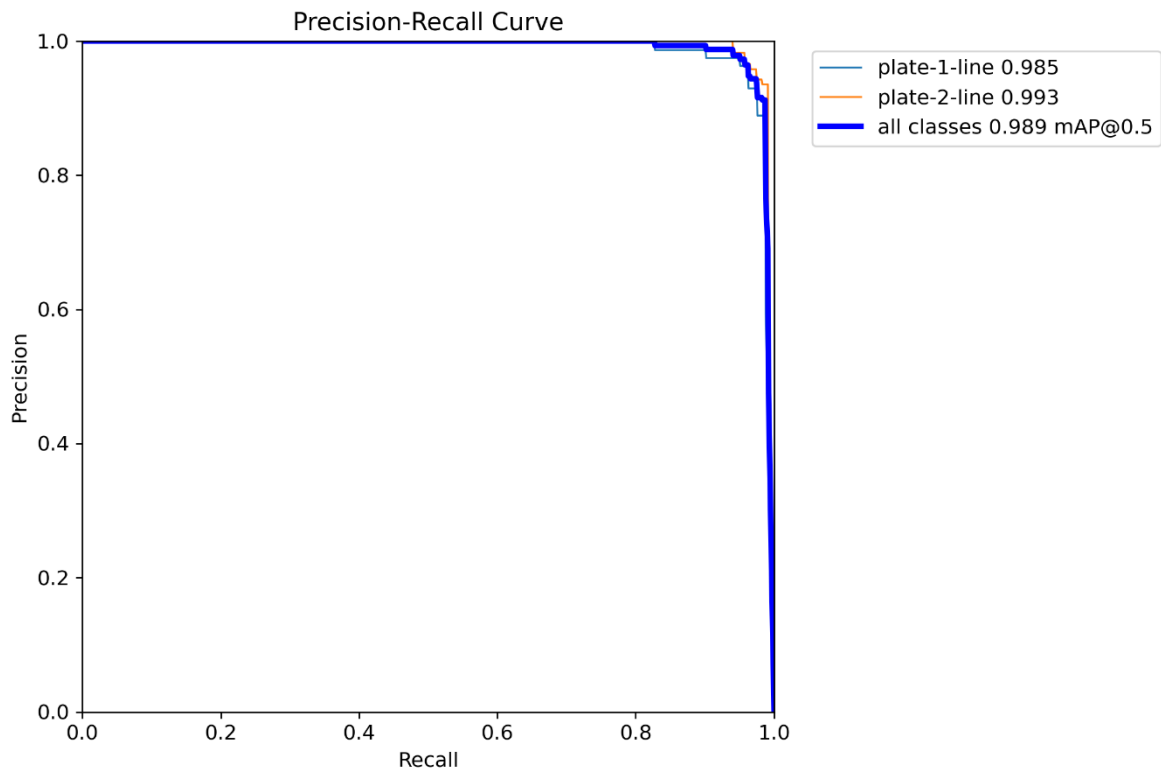
Dựa trên biểu đồ kết quả tổng hợp (Hình 3.1), các hàm mất mát trên cả tập huấn luyện và tập xác thực (train/val loss cho box, pose, cls) đều thể hiện xu hướng giảm tiệm cận và hội tụ tốt ở những epoch cuối, chứng tỏ mô hình không gặp phải hiện tượng học vẹt (Overfitting).



Hình 3.1. Biểu đồ hàm mất mát và các chỉ số mAP trong quá trình huấn luyện

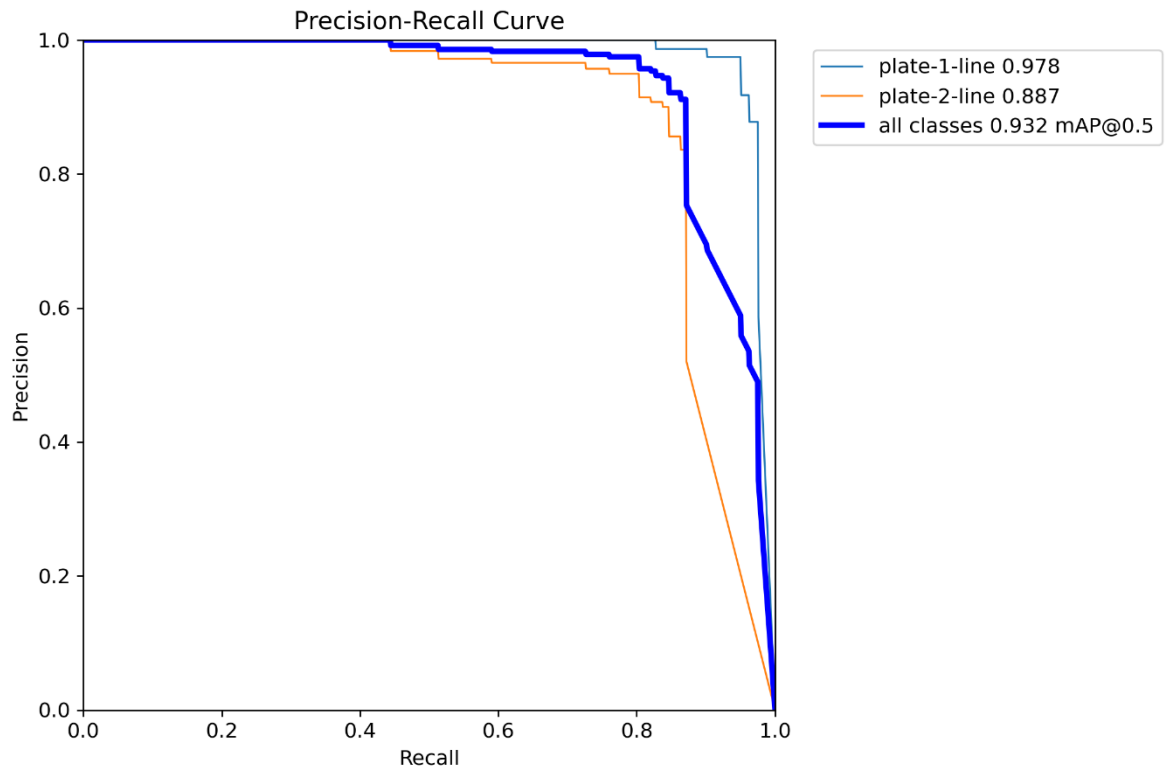
- **Đánh giá khả năng định vị hộp bao (Bounding Box):** Khả năng khoanh vùng chứa biển số của mô hình đạt mức xuất sắc. Theo biểu đồ Precision-Recall (Hình

3.2), mô hình đạt chỉ số **mAP@0.5 tổng thể là 98.9%**. Trong đó, khả năng phát hiện biển số 2 dòng (plate-2-line) đạt 99.3%, nhỉnh hơn đôi chút so với biển số 1 dòng (plate-1-line) đạt 98.5%. Điều này cho thấy mạng nơ-ron đã học được rất tốt các đặc trưng hình khối cơ bản của biển số trong môi trường nhiễu.



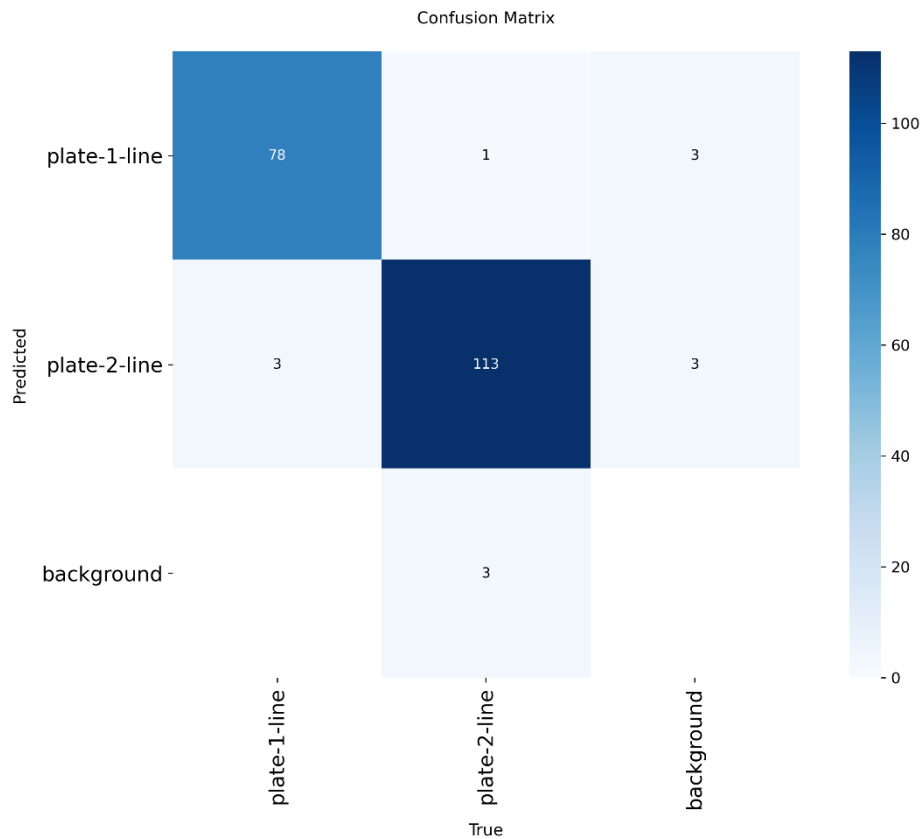
Hình 3.2. Biểu đồ Precision-Recall đánh giá khả năng định vị hộp bao (Box).

- **Đánh giá khả năng dự đoán điểm đặc trưng (Keypoints / Pose):** Tác vụ dự đoán chính xác 4 điểm góc khó hơn rất nhiều so với vẽ hộp bao, do nó đòi hỏi sự hiểu biết về phối cảnh không gian. Dù vậy, mô hình vẫn đạt chỉ số **mAP@0.5 tổng thể ấn tượng ở mức 93.2%** (Hình 3.3). Điểm thú vị là ở tác vụ này, biển số 1 dòng lại đạt độ chính xác nội suy góc tốt hơn (97.8%) so với biển 2 dòng (88.7%). Nguyên nhân thực tế có thể xuất phát từ việc biển 2 dòng (thường gắn trên xe máy) hay bị vướng các đỉnh ốc lớn hoặc bị che khuất một phần bởi thanh chắn bùn, khiến mô hình khó xác định chính xác các điểm giới hạn vật lý.



Hình 3.3. Biểu đồ Precision-Recall đánh giá khả năng dự đoán tọa độ 4 góc (Pose).

- Phân tích Ma trận nhầm lẫn (Confusion Matrix):** Để quan sát chi tiết tỷ lệ dự đoán sai lệch (False Positives / False Negatives), hệ thống trích xuất Ma trận nhầm lẫn của mô hình (Hình 3.4). Kết quả cho thấy tỷ lệ nhận diện nhầm với bối cảnh nền (Background noise) là cực kỳ thấp. Trong toàn bộ tập kiểm thử, chỉ có 6 trường hợp ảnh nền bị dự đoán nhầm thành biển số, và 3 trường hợp biển số thực tế bị bỏ sót (phân loại thành nền). Sự nhầm lẫn giữa hai lớp plate-1-line và plate-2-line cũng rất hiếm gặp (chỉ khoảng 4 trường hợp), minh chứng cho khả năng phân tách đặc trưng cực tốt của lớp phân loại (Classification Head).

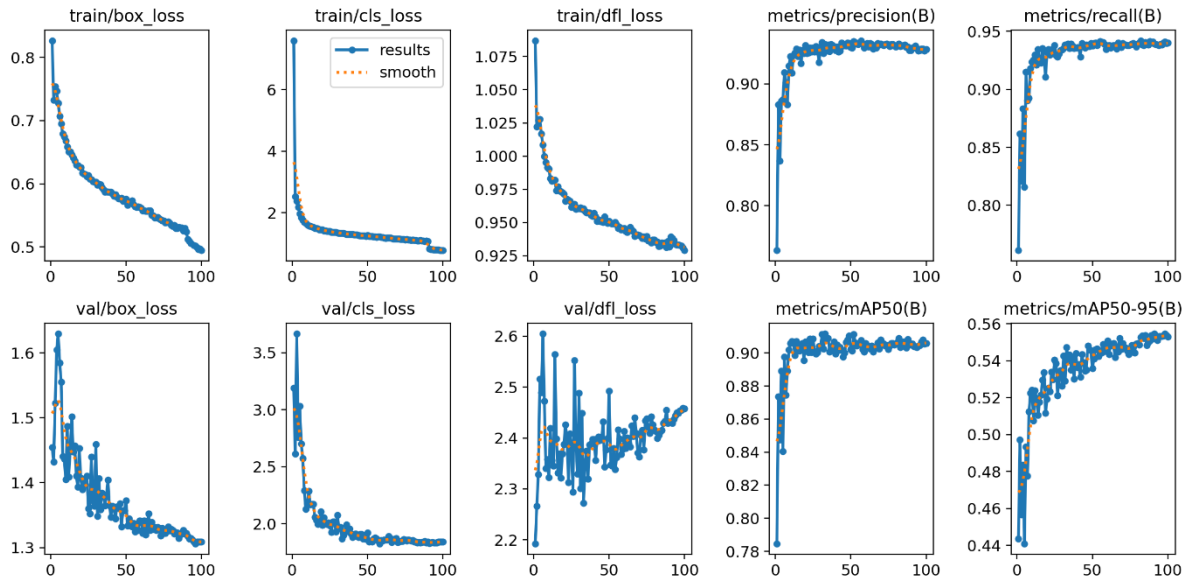


Hình 3.4. Ma trận nhầm lẫn (Confusion Matrix) của mô hình phát hiện biển số.

3.1.3.2. Đánh giá mô hình Nhận diện ký tự và Màu sắc (OCR & Color)

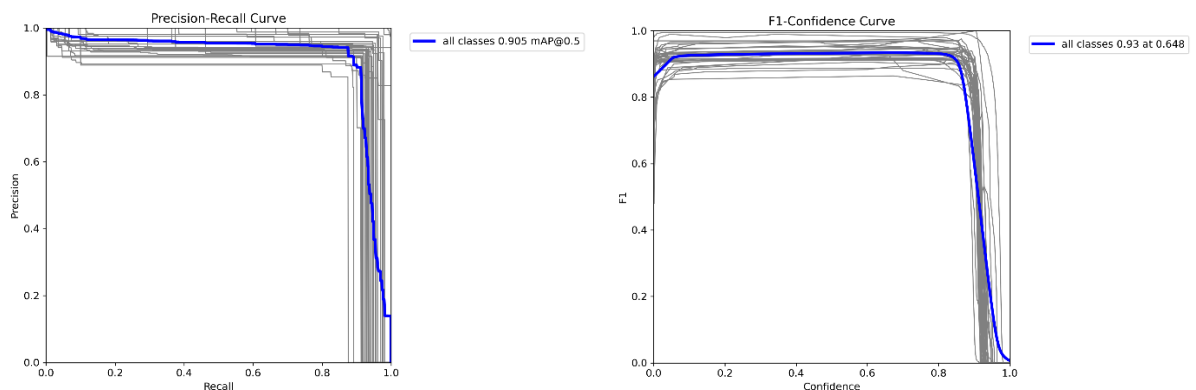
Mô hình YOLO thứ hai trong hệ thống đảm nhận kiến trúc đa nhiệm (Multi-task Learning): vừa trích xuất hộp bao bao quanh từng ký tự riêng lẻ, vừa dự đoán phân loại màu nền của toàn bộ biển số. Khác với mô hình phát hiện biển số chỉ có 2 nhãn (1 dòng, 2 dòng), mô hình này phải phân tách không gian đặc trưng của 35 nhãn (bao gồm 31 ký tự chữ/số và 4 loại màu biển).

Dựa trên biểu đồ huấn luyện (Hình 3.5), hàm mất mát phân loại (val/cls_loss) giảm sâu và ổn định, chứng tỏ mô hình đã học được tốt sự khác biệt giữa các lớp đặc trưng này.



Hình 3.5. Biểu đồ quá trình huấn luyện của mô hình OCR và Màu sắc

- Đánh giá hiệu năng tổng thể (mAP và F1-Score):** Theo biểu đồ Precision-Recall (Hình 3.6 – trái), mặc dù phải đối mặt với số lượng nhãn phân loại lớn và kích thước đối tượng nhỏ (chỉ vài chục pixel cho mỗi ký tự), mô hình vẫn đạt độ chính xác trung bình **mAP@0.5 ở mức 90.5%**. Đáng chú ý, biểu đồ F1-Confidence Curve (Hình 3.6 – phải) cho thấy điểm cân bằng tối ưu giữa Độ chính xác (Precision) và Độ phủ (Recall) đạt **F1 = 93%** tại ngưỡng tin cậy (Confidence) là **0.648**. Kết quả này cung cấp cơ sở khoa học vững chắc để thiết lập cấu hình siêu tham số (ngưỡng `conf_threshold`) cho hệ thống API ở mức 0.6 đến 0.65 nhằm lọc bỏ nhiều hiệu quả nhất.

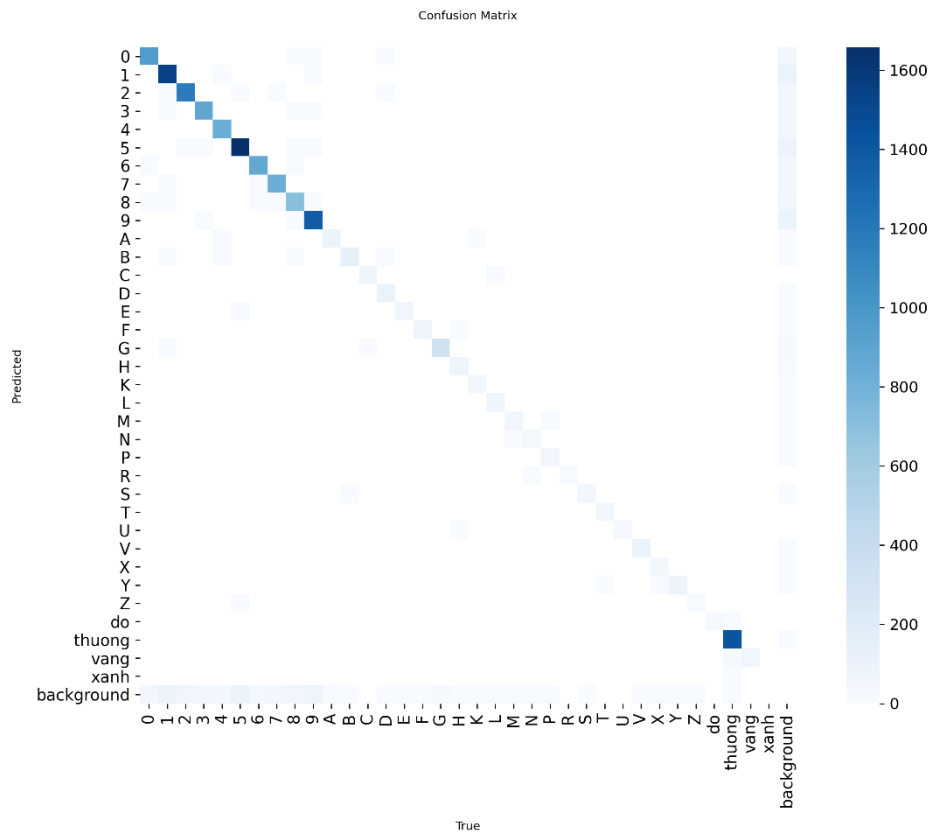


Hình 3.6. Biểu đồ Precision-Recall (trái) và F1-Curve (phải) của mô hình OCR.

- Phân tích Ma trận nhầm lẫn (Confusion Matrix):** Ma trận nhầm lẫn (Hình 3.7) cho thấy một bức tranh trực quan và chân thực về năng lực suy luận cũng như các giới hạn vật lý của mô hình:
 - Tác vụ phân loại màu sắc:** Đường chéo chính tại các nhãn màu (thương - biển trắng, vàng, xanh) có màu xanh rất đậm, chứng tỏ mô hình phân

loại màu nền gần như chính xác tuyệt đối. Lớp biển trắng chiếm ưu thế về số lượng dự đoán đúng do sự phổ biến của tập dữ liệu phân phối thực tế.

- **Tác vụ nhận diện ký tự:** Hầu hết các ký tự chữ cái và chữ số đều có độ hội tụ chính xác cao trên đường chéo chính. Tuy nhiên, ma trận cũng phơi bày một số sai số mang tính đặc thù của bài toán OCR trên biển số. Điển hình là sự nhầm lẫn nhẹ giữa các cặp ký tự có hình thái cấu trúc tương đồng nhau trong điều kiện ảnh mờ hoặc lóa sáng, ví dụ như: **8 và B, D và 0, hoặc G và 6**.
- **Nhiều nền (Background):** Một phần rất nhỏ dữ liệu nền (background) bị nhận diện nhầm thành ký tự. Điều này cho thấy một tỷ lệ rất nhỏ các chi tiết nhiễu trên biển số (như đỉnh ốc, bản, ...) thỉnh thoảng bị mô hình "ảo giác" và nhận diện nhầm thành ký tự. Tuy nhiên, tỷ lệ này là không đáng kể và hoàn toàn có thể bị triệt tiêu ở bước hậu xử lý nhờ vào việc tinh chỉnh ngưỡng tin cậy.



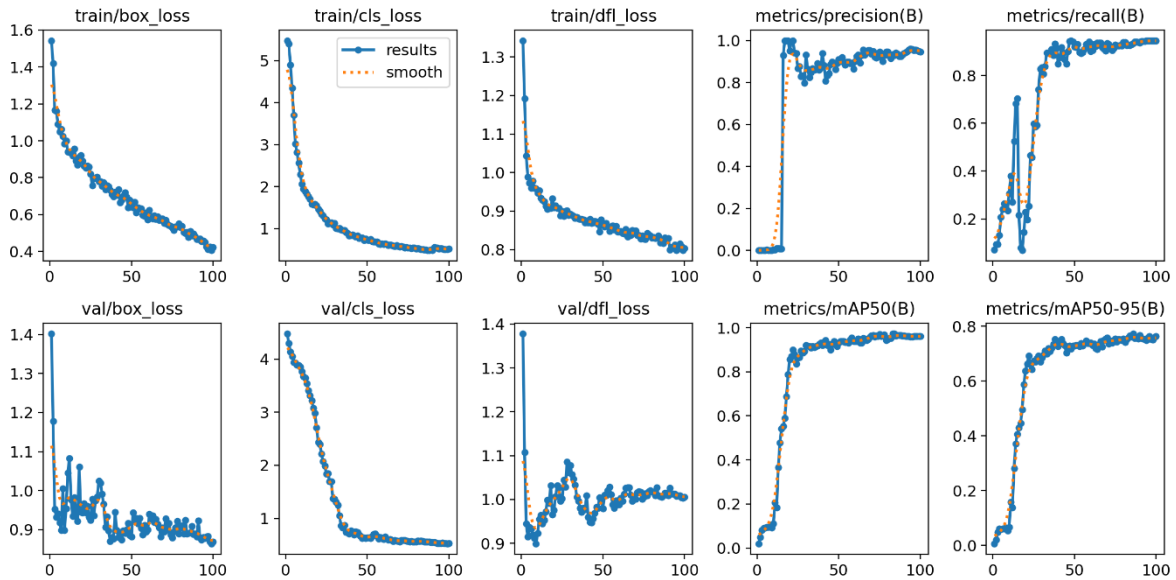
Hình 3.7. Ma trận nhầm lẫn chi tiết các lớp ký tự và màu nền biển số.

Nhận xét: Việc ứng dụng mạng phát hiện đối tượng (Object Detection) thay cho các mạng phân tích chuỗi thuần túy (như CRNN) đã mang lại kết quả tích cực, giúp hệ thống không chỉ đọc được chữ mà còn định vị được không gian phân bố của chữ, tạo tiền đề cho thuật toán hậu xử lý (sắp xếp chuỗi theo dòng) hoạt động hiệu quả. Những sai số

nhỏ lẻ giữa các ký tự đồng dạng có thể được khắc phục bằng các thuật toán kiểm tra tính hợp lệ của định dạng biến số (Regex).

3.1.3.3. Đánh giá mô hình Phân loại phương tiện

Mô hình thuộc Luồng 1 của hệ thống đóng vai trò khoanh vùng và phân loại các phương tiện giao thông (xe máy, ô tô) cũng như nhận diện chi tiết thương hiệu (VinFast, Toyota, Mazda...). Nhìn vào biểu đồ quá trình huấn luyện (Hình 3.8), hàm mất mát phân loại (train/cls_loss và val/cls_loss) thể hiện sự hội tụ rất tốt và đồng pha, chứng tỏ mạng nơ-ron nắm bắt rất nhanh các đặc trưng thương hiệu. Tuy nhiên, hàm mất mát định vị hộp bao trên tập xác thực (val/box_loss) lại có dấu hiệu chững lại và dao động nhẹ từ sau epoch thứ 40. Hiện tượng quá khớp (overfitting) cục bộ này ở khâu vẽ hộp bao là hoàn toàn dễ hiểu do giới hạn về quy mô tập dữ liệu thử nghiệm (~400 ảnh), song nó không làm ảnh hưởng đến năng lực phân loại tổng thể của luồng hệ thống.

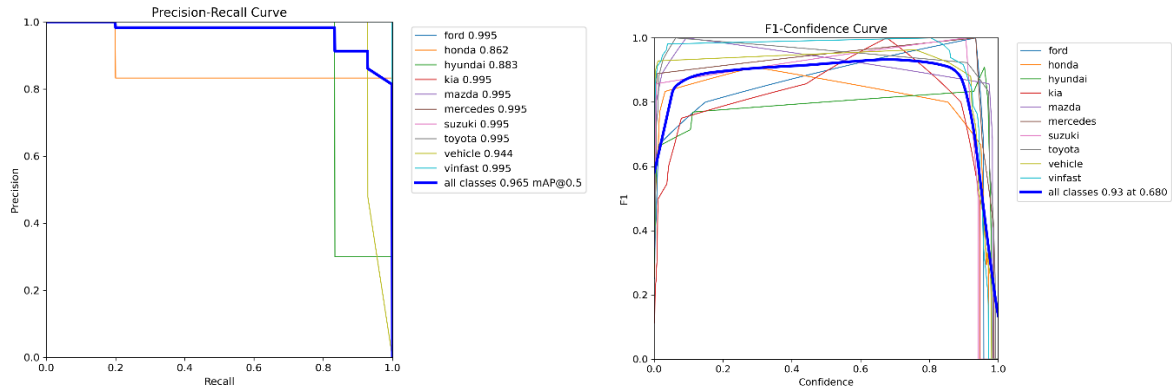


Hình 3.8. Biểu đồ quá trình huấn luyện của mô hình phân loại phương tiện

Nhận xét về giới hạn dữ liệu: trọng tâm cốt lõi của đề án là chuỗi bài toán Nhận dạng biển số tự động (ALPR). Tác vụ Phân loại phương tiện (Luồng 1) được tích hợp thêm dưới dạng một mô-đun thử nghiệm tính khả thi (Proof of Concept / Demo) nhằm minh chứng cho khả năng chạy đa luồng song song và tính dễ dàng mở rộng của kiến trúc API. Do đó, với tập dữ liệu khởi điểm khoảng 400 ảnh, việc mô hình đôi khi xuất hiện dao động nhẹ hoặc nhạy cảm với nhiễu nền ở các thương hiệu xe hiếm gặp là điều hoàn toàn nằm trong dự tính. Mô hình hiện tại đã hoàn thành xuất sắc vai trò "kiểm chứng luồng hệ thống". Việc nâng cấp độ chính xác toàn diện cho mô-đun này sẽ được đặt làm định hướng phát triển trong tương lai khi có điều kiện mở rộng quy mô thu thập dữ liệu.

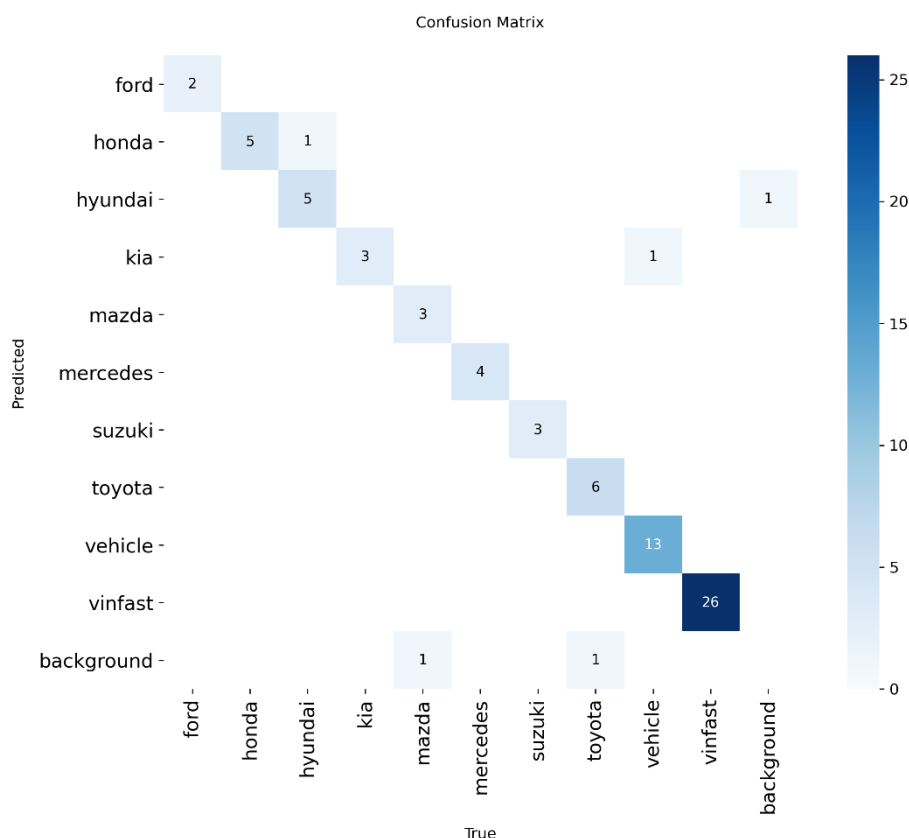
- **Đánh giá hiệu năng tổng thể (mAP và F1-Score):** Bất chấp giới hạn về quy mô dữ liệu, mô hình vẫn chất lọc được các đặc trưng quan trọng nhất. Biểu đồ Precision-Recall (Hình 3.9 - trái) ghi nhận chỉ số **mAP@0.5 tổng thể đạt mức**

xuất sắc 96.5%. Nhiều thương hiệu phổ biến (như VinFast, Kia, Mazda, Toyota) đạt mAP lên tới 99.5%. Đường cong F1-Confidence Curve (Hình 3.9 - phải) cũng chỉ ra rằng mô hình đạt độ cân bằng tối ưu **F1 = 93%** tại ngưỡng tin cậy **0.680**. Đây là cơ sở để thiết lập tham số bộ lọc tự động trong tệp vehicle.py, giúp loại bỏ các dự đoán có độ tin cậy thấp.



Hình 3.9. Biểu đồ Precision-Recall (trái) và F1-Curve (phải) của mô hình Phân loại phương tiện.

- Phân tích Ma trận nhầm lẫn (Confusion Matrix):** Sự phân tách lớp đặc trưng của mô hình được thể hiện rất rõ nét thông qua Ma trận nhầm lẫn (Hình 3.10). Đường chéo chính sắc nét cho thấy mô hình phân biệt rất chính xác giữa lớp phương tiện chung (vehicle - đại diện chủ yếu cho xe máy) và các thương hiệu ô tô cụ thể (đặc biệt là nhãn VinFast với 26 mẫu dự đoán đúng tuyệt đối). Có một vài trường hợp sai số rất nhỏ lẻ (ví dụ: 1 chi tiết bối cảnh nền bị đoán nhầm thành xe Hyundai, hoặc xe Mazda/Toyota bị mô hình bỏ sót phán đoán thành nền). Tuy nhiên, đối với hệ thống Smart Parking tổng thể, nhánh phân loại thương hiệu này hiện đang được tích hợp ở mức độ thử nghiệm (Proof of Concept) nhằm chứng minh khả năng mở rộng tính năng. Sự nhầm lẫn nhỏ lẻ giữa các hãng xe hoàn toàn không làm ảnh hưởng đến luồng trích xuất biên số cốt lõi ở nhánh phía sau, do đó rủi ro này nằm trong biên độ an toàn và hoàn toàn chấp nhận được.



Hình 3.10. Ma trận nhầm lẫn của mô hình Phân loại phương tiện.

3.2. Tối ưu hóa hiệu năng suy luận bằng Intel OpenVINO

Mặc dù các mô hình YOLO đã đạt được độ chính xác rất cao sau quá trình huấn luyện, nhưng tệp trọng số nguyên bản của PyTorch (định dạng .pt) thường có dung lượng lớn và chứa nhiều phép toán đồ thị phức tạp. Điều này khiến mô hình vận hành chậm và tiêu tốn nhiều tài nguyên khi triển khai trên các thiết bị chỉ có vi xử lý trung tâm (CPU). Để giải quyết bài toán hiệu năng thời gian thực cho API, toàn bộ 3 mô hình (Vehicle, Plate-Pose, OCR) đều được đưa qua một pipeline (luồng) tối ưu hóa bằng bộ công cụ Intel OpenVINO trước khi đưa vào vận hành.

3.2.1. Trích xuất đồ thị tính toán sang chuẩn ONNX

Bước đầu tiên trong chu trình tối ưu là tách rời mạng nơ-ron khỏi sự phụ thuộc vào framework PyTorch. Trọng số tốt nhất (best.pt) của các mô hình được xuất sang định dạng ONNX (Open Neural Network Exchange). Quá trình này được cấu hình với các tham số chiến lược:

- **Cố định không gian đầu vào (Static Input Shape):** Kích thước ảnh được chốt cứng ở mức 640x640. Việc cố định tensor đầu vào giúp loại bỏ các phép toán cấp phát bộ nhớ động thừa thãi trong quá trình suy luận.

- **Tương thích toán hạng (Opset 12):** Sử dụng bộ toán hạng (Opset) phiên bản 12 để đảm bảo sự tương thích hoàn hảo và ổn định nhất khi biên dịch các hàm kích hoạt phức tạp của YOLO sang ngôn ngữ máy.
- **Đơn giản hóa đồ thị (Graph Simplification):** Đây là một kỹ thuật can thiệp sâu. Hệ thống sẽ tự động rà soát kiến trúc mạng để gộp (fuse) các lớp tính toán tuyến tính đứng liền nhau (ví dụ: gộp lớp Tích chập - Convolution và lớp Chuẩn hóa lô - Batch Normalization thành một ma trận duy nhất). Thao tác này giúp giảm đáng kể số lượng phép toán dấu phẩy động (FLOPs) mà không làm suy giảm độ chính xác.

3.2.2. Biên dịch sang định dạng trung gian (OpenVINO IR)

Mô hình ONNX tiếp tục được nạp vào lõi của OpenVINO (ov.Core()) để thực hiện bước biên dịch cuối cùng. Đồ án xây dựng một luồng mã hóa (Script Pipeline) tự động chuyển đổi mô hình từ ONNX sang định dạng Trung gian - IR (Intermediate Representation). Kết quả đầu ra của quá trình này là một bộ đôi tệp tin siêu nhẹ:

- **Tệp .xml:** Chứa cấu trúc mạng (Network Topology) đã được tối ưu hóa cho tập lệnh vi xử lý của Intel.
- **Tệp .bin:** Chứa các tham số trọng số (Weights) và độ lệch (Biases) ở dạng nhị phân, hỗ trợ khả năng nạp nhanh trực tiếp vào bộ nhớ đệm của CPU.

3.2.3. Tối ưu hóa cấu hình Runtime (Cơ chế Latency)

Ngoài việc tối ưu hóa cấu trúc mô hình ở mức trọng số, hệ thống còn được can thiệp vào tầng thực thi của OpenVINO Runtime để định hình hành vi xử lý dữ liệu. Trong các module nhận diện, cấu hình PERFORMANCE_HINT được thiết lập ở giá trị LATENCY.

Ý nghĩa kỹ thuật: Khác với thiết lập THROUGHPUT vốn ưu tiên xử lý song song nhiều luồng dữ liệu để đạt số lượng khung hình trên giây (FPS) cao nhất, thiết lập LATENCY ép bộ vi xử lý tập trung tối đa tài nguyên luồng (threads) để giải quyết dứt điểm một đơn vị hình ảnh trong thời gian ngắn nhất.

Tính phù hợp: Đối với đặc thù của hệ thống ALPR triển khai tại các trạm kiểm soát hoặc bãi đỗ xe, việc phản hồi kết quả tức thời cho từng phương tiện quan trọng hơn việc xử lý hàng loạt. Lựa chọn này giúp giảm thiểu rủi ro mất dấu đối tượng khi phương tiện di chuyển nhanh qua vùng quan sát của camera, đảm bảo tính ứng dụng thực tế của đồ án trên các phần cứng phổ thông không trang bị GPU rời.

3.3. Triển khai API Backend và Đóng gói Container

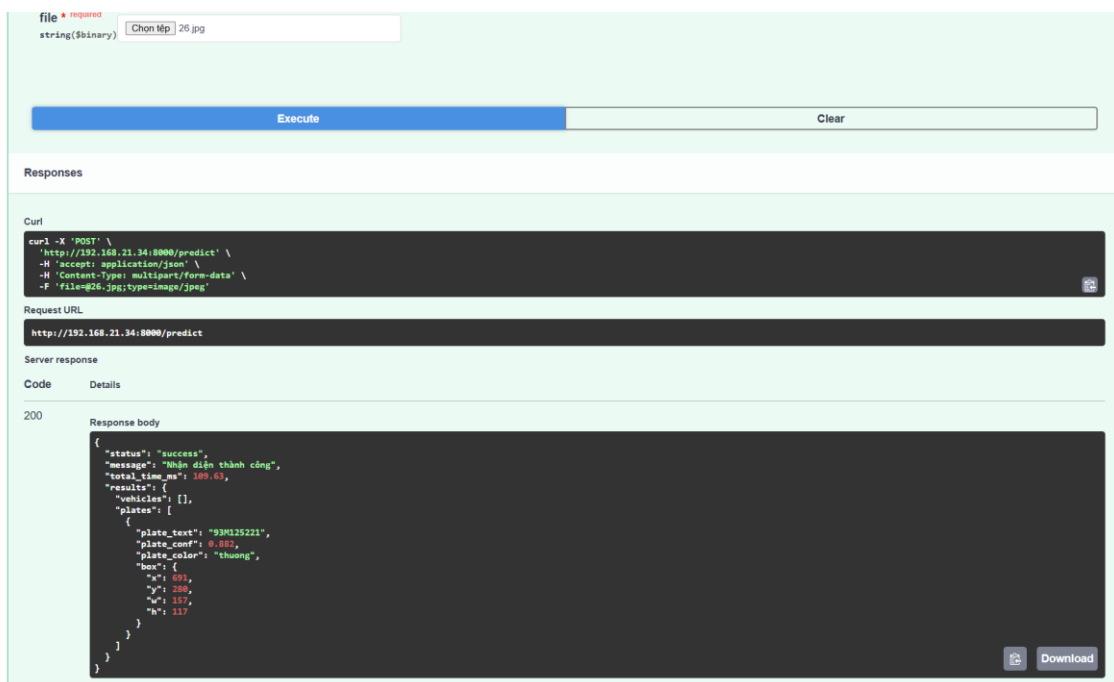
Để chuyển đổi các mô hình trí tuệ nhân tạo từ môi trường nghiên cứu sang một sản phẩm phần mềm thực thụ, hệ thống cần được đóng gói bền vững. Giai đoạn triển khai này tập

trung vào hai nhiệm vụ cốt lõi: Tích hợp mã nguồn thành dịch vụ API và Đóng gói toàn bộ môi trường thực thi bằng công nghệ Docker.

3.3.1. Tích hợp mã nguồn và Xây dựng dịch vụ API

Hệ thống được xây dựng dưới dạng một dịch vụ Web hiệu năng cao sử dụng framework FastAPI. Đây là tầng trung gian chịu trách nhiệm điều phối dòng dữ liệu giữa người dùng và các mô hình AI lõi.

- **Cấu trúc Endpoint:** Hệ thống cung cấp endpoint chính /predict với phương thức POST, cho phép nhận dữ liệu hình ảnh dưới dạng multipart/form-data.
- **Cơ chế xử lý song song:** Tận dụng tối đa khả năng xử lý bất đồng bộ của Python, tệp api.py sử dụng hàm asyncio.gather kết hợp với run_in_threadpool. Cơ chế này cho phép hệ thống ném đồng thời hai tác vụ nặng là nhận diện phương tiện (thông qua Class VehicleDetector) và phát hiện biển số (thông qua Class PlateDetector) vào nhóm luồng (Threadpool) để thực thi song song ngay khi dữ liệu hình ảnh được giải mã.
- **Module hóa mã nguồn:** Các mô hình AI được thiết kế thành các lớp độc lập trong các tệp detector.py, ocr.py và vehicle.py. Cách tiếp cận này giúp mã nguồn API trở nên tinh gọn, dễ bảo trì và cho phép các luồng xử lý (như OCR từ Class YOLORecognizer) có thể được gọi một cách linh hoạt sau khi có kết quả từ tầng phát hiện biển số.
- **Giao diện thử nghiệm:** FastAPI tự động khởi tạo giao diện Swagger UI, cho phép lập trình viên kiểm thử các tham số đầu vào và quan sát cấu trúc JSON trả về một cách trực quan mà không cần thông qua các công cụ bên thứ ba.



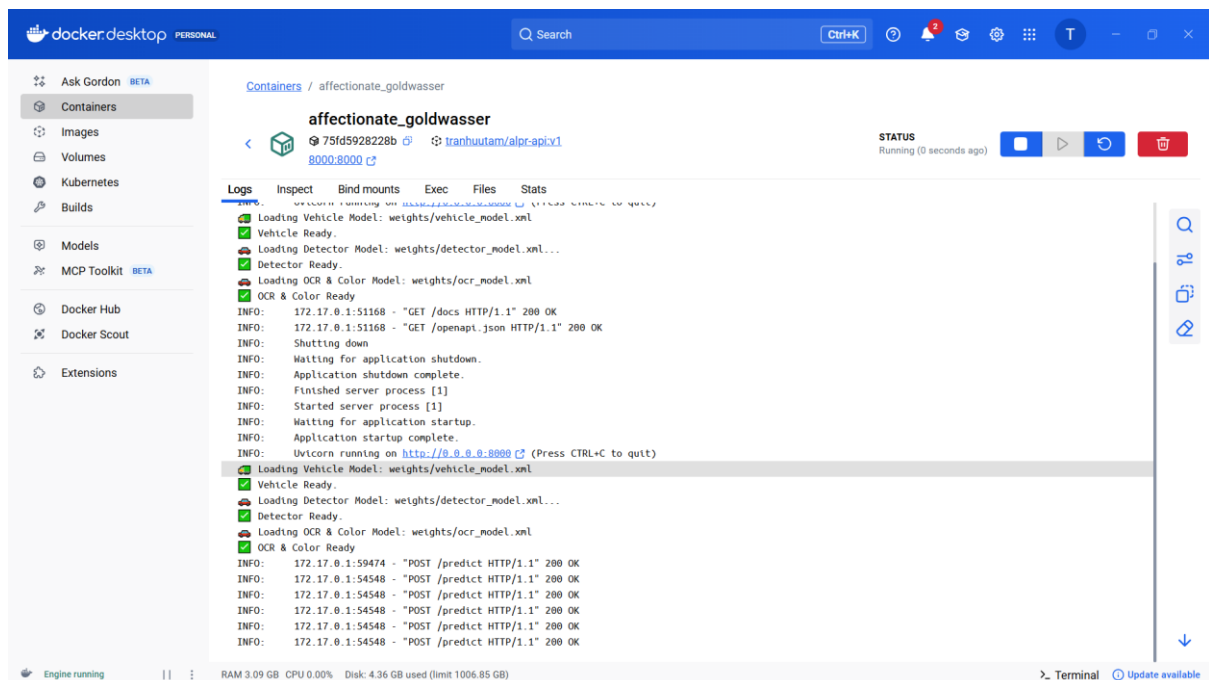
Hình 3.11. Giao diện Swagger UI của hệ thống

(Ghi chú: Hệ thống phản hồi chính xác biến số "93M125221" với tổng thời gian xử lý (Latency) chỉ 109.63ms trên môi trường CPU).

3.3.2. Đóng gói ứng dụng với công nghệ Docker

Nhằm đảm bảo khả năng chạy độc lập và loại bỏ hoàn toàn lỗi "xung đột môi trường", hệ thống được đóng gói thành một Docker Image hoàn chỉnh thông qua tệp cấu hình Dockerfile.

- **Tối ưu hóa môi trường:** Hệ thống sử dụng **image base** python:3.11-slim [3] để giảm thiểu dung lượng image và bộ nhớ. Các thư viện hệ thống thiết yếu cho xử lý ảnh như libgl1 (OpenCV) và libglib2.0-0 được cài đặt trực tiếp trong quá trình xây dựng image.
- **Quy trình vận hành:** Toàn bộ mã nguồn, bộ trọng số mô hình đã tối ưu bằng OpenVINO và các thư viện phụ thuộc được "đóng gói" vào trong container. Khi khởi chạy, hệ thống sẽ tự động thực hiện quá trình nạp mô hình (Warm-up) vào bộ nhớ và ứng dụng sẽ tự động lắng nghe trên cổng 8000 thông qua máy chủ ASGI Uvicorn.
- **Tính linh hoạt:** Kết quả triển khai thực tế trên Docker Desktop cho thấy hệ thống vận hành ổn định. Các dòng nhật ký hệ thống (Logs) xác nhận việc nạp thành công các bộ trọng số tối ưu từ OpenVINO cho cả 3 mô hình: Vehicle Detector, Plate Detector và OCR. Điều này đảm bảo môi trường thực thi giống hệt 100% giữa máy lập trình và máy chủ thực tế.



Hình 3.12. Container hóa ứng dụng trên Docker

(Ghi chú: Nhật ký hệ thống (Logs) xác nhận việc nạp thành công các bộ trọng số tối ưu và sẵn sàng tiếp nhận request).

3.4. Đánh giá hệ thống tổng thể

Để đo lường hiệu năng thực tiễn, đồ án tiến hành kiểm thử trên tập dữ liệu gồm **6.930 ảnh**. Quá trình này phản ánh năng lực phối hợp giữa mô hình YOLO-Pose, thuật toán Perspective Transform và mô hình YOLO-OCR trong các kịch bản trên thực tế.

3.4.1. Đánh giá định lượng

Để phản ánh khách quan nhất năng lực thực tiễn của hệ thống, tập dữ liệu kiểm thử là sự hỗn hợp ngẫu nhiên của nhiều kịch bản trên thực tế (từ môi trường lý tưởng, ban ngày cho đến các trường hợp phức tạp như lóa đèn pha ban đêm, biển số xô lệch hoặc bám bẩn). Quá trình kiểm thử áp dụng tiêu chí đánh giá sau:

- **Tỉ lệ phát hiện:** Đánh giá năng lực của khối YOLO-Pose. Một trường hợp được tính là thành công khi hệ thống định vị được hộp bao (Bounding Box) chứa biển số và trích xuất đủ 4 điểm đặc trưng (Keypoints) để đưa vào nắn phẳng.
- **Độ chính xác toàn chuỗi (End-to-End Accuracy):** Đánh giá năng lực tổng thể. Một biển số chỉ được tính là nhận diện đúng khi mô hình đọc chính xác 100% toàn bộ các ký tự và sắp xếp đúng thứ tự cấu trúc. Sai lệch dù chỉ một ký tự được tính là thất bại hoàn toàn.
- **Tiêu chuẩn kiểm chứng:** Nhân gốc (Ground Truth) được đối chiếu dựa trên khả năng nhận thức của mắt thường. Trong một số trường hợp cực đoan (ảnh nhòe do chuyển động nhanh, lóa sáng, che khuất, ...), nếu mắt người không thể nhìn rõ chữ để đối chiếu, kết quả đó không được tính là chính xác.

Dựa trên các tiêu chí trên, kết quả kiểm thử định lượng được tổng hợp tại Bảng 3.1:

Bảng 3.1. Kết quả kiểm thử tổng quan của mô hình

Tiêu chí đánh giá	Kết quả	Ý nghĩa
Tổng số ảnh kiểm thử	6930	Dữ liệu hỗn hợp môi trường thực tế.
Số ảnh phát hiện đúng vùng biển	6.907	Khối YOLO-Pose bắt được hộp bao và 4 góc.
Tỉ lệ phát hiện (Detection Rate)	99,67%	Độ chính xác mô hình phát hiện biển số.
Số ảnh nhận diện đúng toàn chuỗi	6439	Khối YOLO-OCR đọc đúng 100% các ký tự.
Tỉ lệ chính xác toàn chuỗi	92.91%	Độ chính xác của toàn bộ hệ thống API.
Thời gian xử lý trung bình	~90.76ms	Độ trễ đo lường trên phần cứng vi xử lý CPU.

Nhận xét:

- **Phạm vi thực nghiệm:** Nhằm tập trung đánh giá năng lực của thuật toán cốt lõi, tiến trình thực nghiệm định lượng trên tập 6.930 ảnh này chỉ tập trung phân tích

các chỉ số của luồng nhận diện biển số (ALPR Pipeline). Tác vụ Phân loại phương tiện (dù chạy song song) đã được đánh giá hiệu năng độc lập ở mục 3.1.3.3 nên không đưa vào bảng đo lường này.

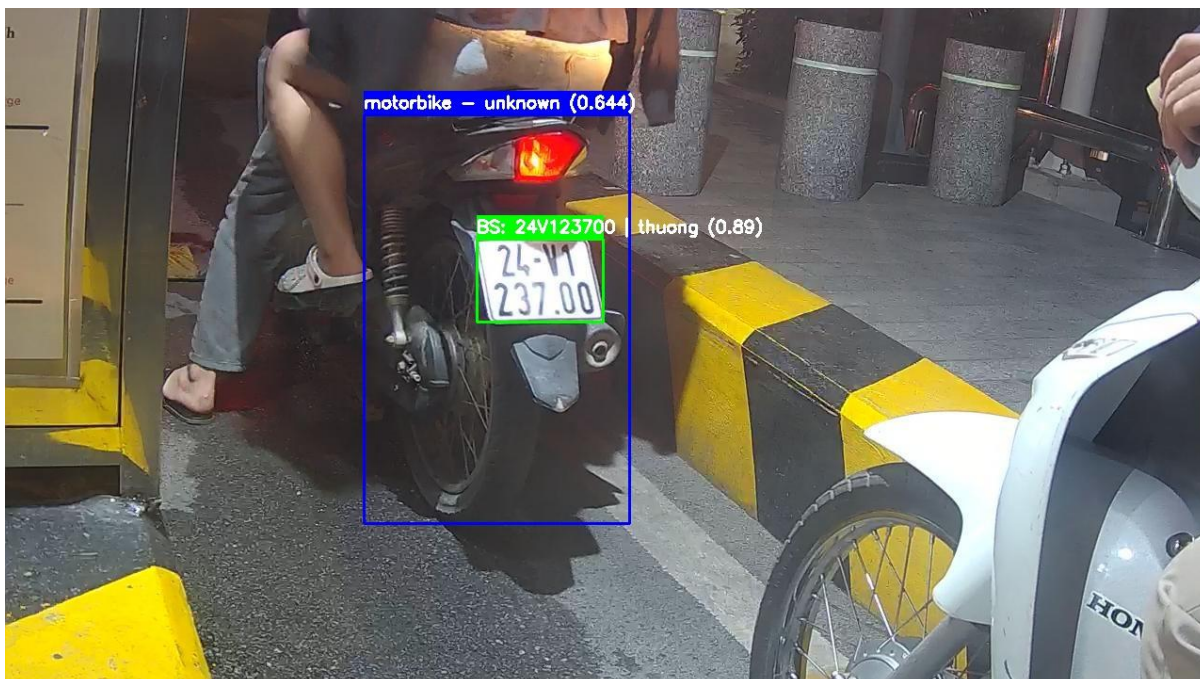
- **Phát hiện biển số:** Tỷ lệ phát hiện đạt mức gần như tuyệt đối (**99,67%**). Trong tổng số 6.930 ảnh, hệ thống chỉ bỏ lọt **23 trường hợp** không thể tìm thấy biển số. Qua phân tích, các ca trượt này rơi vào những tình huống giới hạn vật lý: biển số bị khuất một phần, biển số bị nhòe, góc chụp chéo, ... Điều này chứng tỏ mô hình hoạt động tốt và ổn định.
- **Nhận diện ký tự:** Tỷ lệ chính xác toàn chuỗi đạt 92.91%. Qua phân tích tập các trường hợp nhận diện sai (~491 ảnh), các lỗi chủ yếu phân bổ vào các nhóm nguyên nhân khách quan:
 - Đọc thừa/thiếu ký tự do định ốc cố định biển bị mô hình nhận diện nhầm thành dấu chấm hoặc chữ số.
 - Sai lệch hình thái ký tự do biển số bám bùn đất hoặc bong tróc sơn (ví dụ: chữ F bị đọc thành E, số 8 thành chữ B).
 - Các trường hợp lóa sáng nặng từ đèn pha ban đêm khiến cấu trúc chữ bị mất hoàn toàn, ngay cả mắt người cũng không thể đối chiếu.
 - Mặc dù có hao hụt, con số 92.91% trên một tập dữ liệu thực tế nhiều nhiễu vẫn là một kết quả cực kỳ khả quan, đáp ứng tốt yêu cầu vận hành của các hệ thống Smart Parking.
- **Hiệu năng:** Với thời gian phản hồi trung bình ~90ms trên môi trường CPU phổ thông, hệ thống đã chứng minh được tính khả thi vượt trội, đáp ứng tốt yêu cầu xử lý ở trên thực tế.

3.4.2. Đánh giá định tính

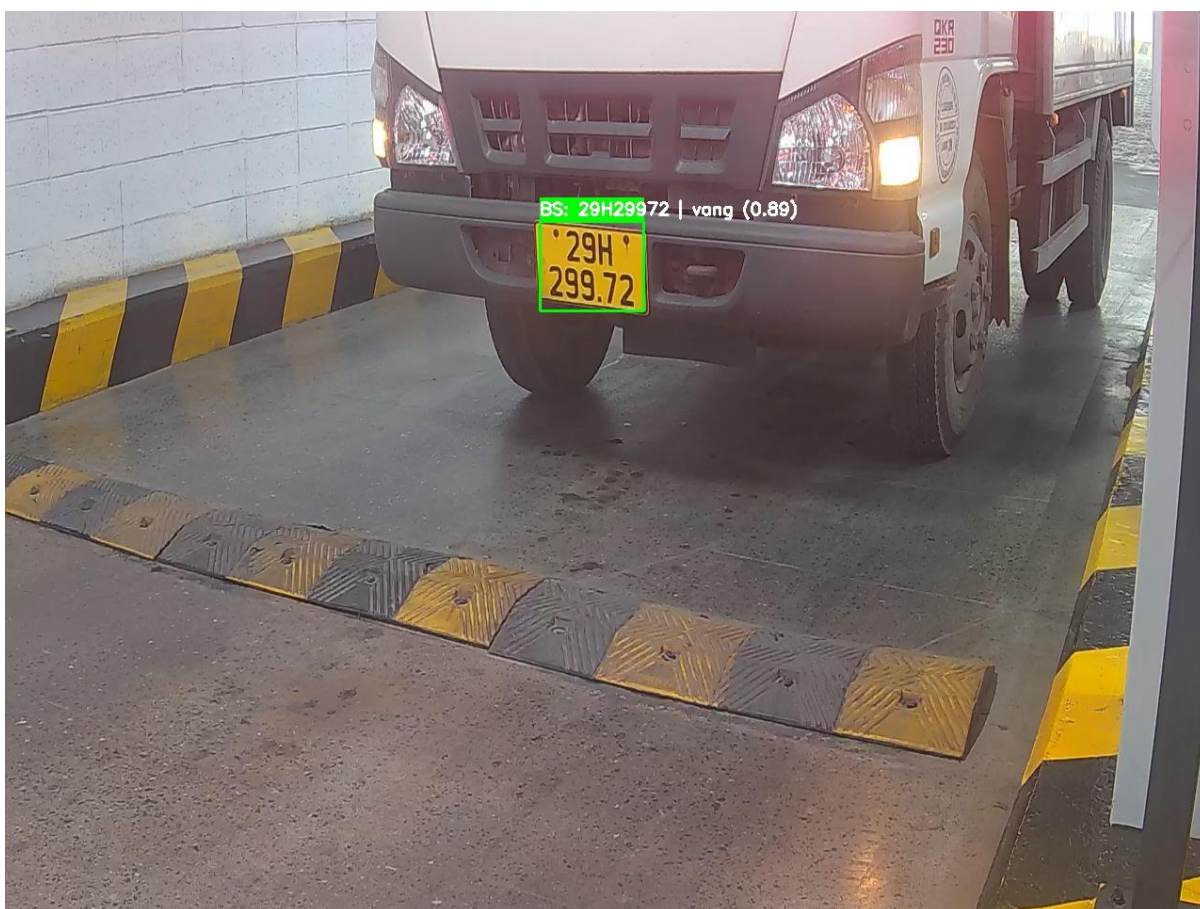
Nhằm cung cấp cái nhìn trực quan về năng lực và giới hạn của hệ thống, một số mẫu kiểm thử điển hình được phân tích chi tiết theo từng nhóm điều kiện môi trường.

Trường hợp 1: Điều kiện môi trường lý tưởng

Trong điều kiện ánh sáng đầy đủ, camera chụp hình ở góc độ trực diện, hệ thống thể hiện sự ổn định tuyệt đối. Các Bounding Box bắt trọn vẹn khu vực biển số; thuật toán OCR phân tách rõ ràng từng ký tự, phân biệt chính xác màu nền và không gặp khó khăn trong việc sắp xếp thứ tự chuỗi.



Hình 3.13. Kết quả nhận diện xe máy trong điều kiện lý tưởng
(Biển số dự đoán: 24V123700 – Biển số đúng: 24V123700)

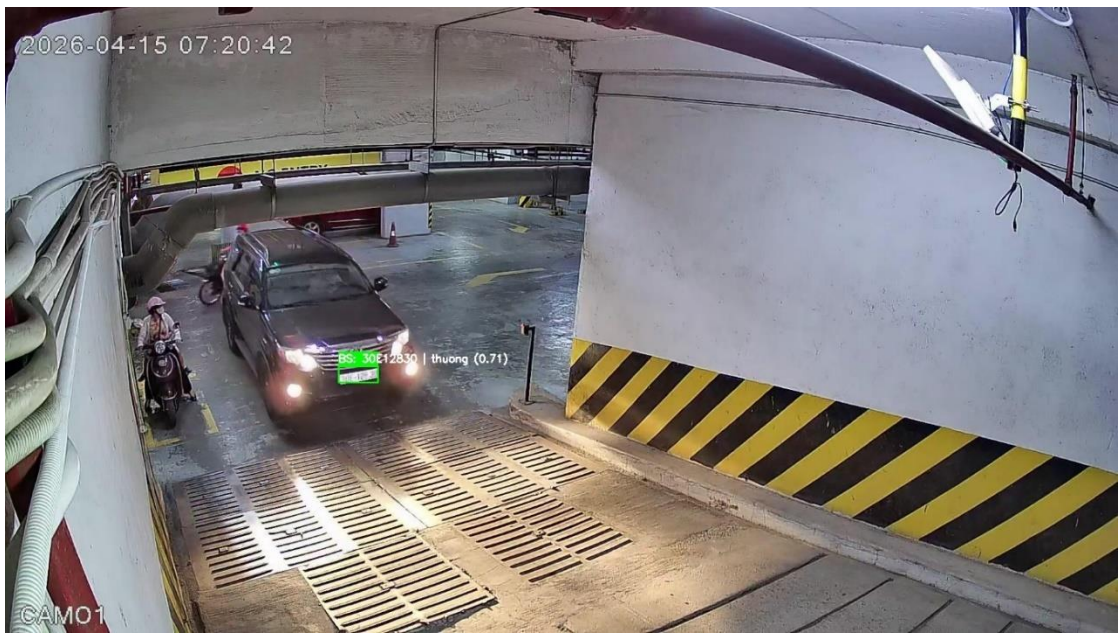


Hình 3.14. Kết quả nhận diện ô tô trong điều kiện lý tưởng
(Biển số dự đoán: 29H29972 – Biển số đúng: 29H29972)

Đặc biệt, như quan sát tại Hình 3.13, bên cạnh việc đọc xuất sắc biển số, API cũng đồng thời phản hồi song song kết quả của luồng Phân loại phương tiện (nhận diện thành công đối tượng 'motorbike'). Điều này là minh chứng trực quan nhất cho năng lực xử lý đa luồng bất đồng bộ của hệ thống, các tác vụ AI chạy song song mà không hề gây tắc nghẽn lẫn nhau.

Trường hợp 2: Điều kiện thực tế phức tạp

Bao gồm các yếu tố gây nhiễu môi trường như: camera đặt ở góc chéo cao/xa gây xô lệch hình học, ảnh chụp ban đêm bị lóa sáng (glare) từ đèn pha, biển số xe bị che khuất một phần,



*Hình 3.15. Kết quả nhận diện trong điều kiện bị lóa sáng + xa
(Biển số dự đoán: 30E12830 – Biển số đúng: 30E12830)*



Hình 3.16. Kết quả nhận diện trong điều kiện góc chụp bị che khuất
(Biển số dự đoán: 29S91701 – Biển số đúng: 29S91701)

Trường hợp 3: AI vượt ngưỡng thị giác người

Đây là một hiện tượng rất thú vị được ghi nhận. Ở các mẫu này, bằng mắt thường, con người gần như không thể dịch được chính xác nội dung biển số. Tuy nhiên, luồng nhận diện của hệ thống vẫn xuất ra các chuỗi ký tự hoàn chỉnh:



Hình 3.17. Khả năng nhận diện vượt ngưỡng khi ở góc xa
(Biển số dự đoán: 90M7221 – ở trường hợp này, ký tự đầu tiên mắt thường không nhìn rõ được là số nào, còn các ký tự sau đều đúng)



Hình 3.18. Khả năng nhận diện vượt ngưỡng khi biển bị che khuất

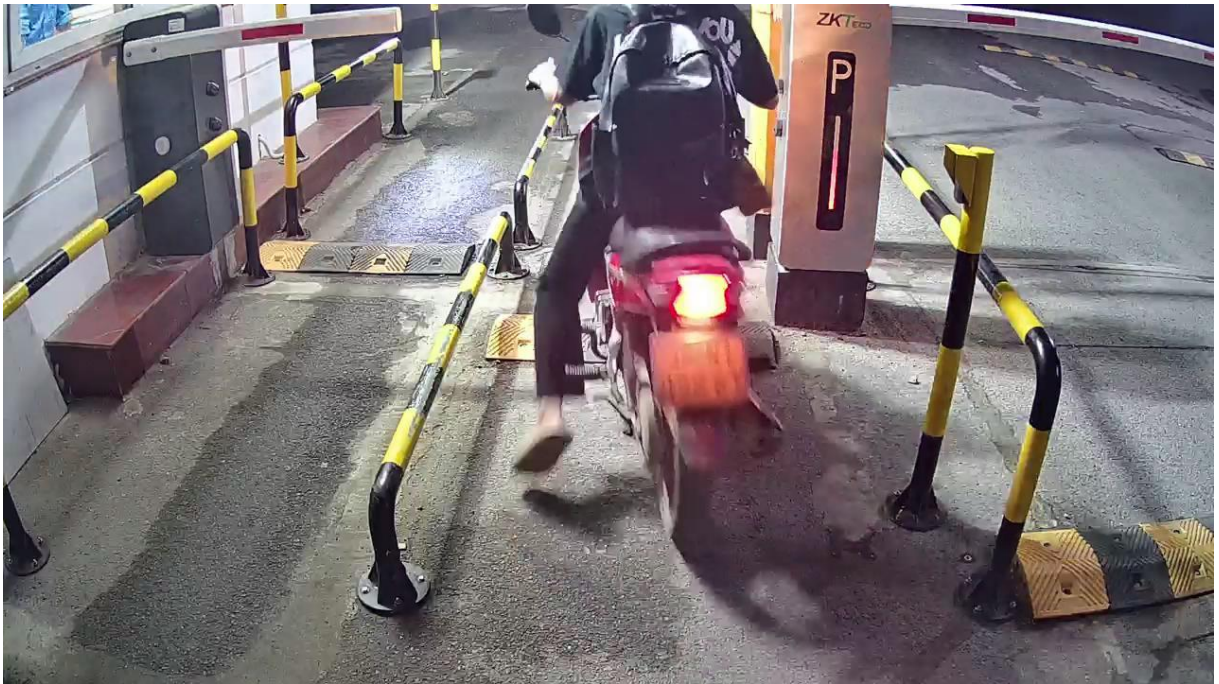
(Biển số dự đoán: 30K36738 – ở trường hợp này 2 ký tự đầu bị che khuất gần hết, mắt thường rất khó quan sát)

Về mặt học thuật, hiện tượng này có thể được lý giải như sau: mạng nơ-ron tích chập (CNN) có khả năng trích xuất các dải gradient ánh sáng ở mức độ vi mô mà mắt người không thấy được. Nhờ đó, mô hình có thể đưa ra dự đoán trên một vùng pixel bị nhiễu nặng dựa trên sự tương đồng với các nhãn đã học.

Trường hợp 4: Một số trường hợp nhận diện sai

Bên cạnh những thành công, hệ thống vẫn tồn tại những giới hạn vật lý dẫn đến các ca nhận diện thất bại. Các sai số này chủ yếu tập trung vào hai nhóm nguyên nhân:

- **Lỗi bỏ sót vùng biển từ khối YOLO-Pose:** Đây là nhóm lỗi xảy ra ở ngay khâu đầu tiên của Pipeline, khiến hệ thống hoàn toàn bỏ lọt đối tượng (chiếm 23 trường hợp trong tập test). Nguyên nhân chủ yếu đến từ các giới hạn môi trường cực đoan làm phá vỡ hoàn toàn đặc trưng hình học của biển số.



Hình 3.19. YOLO-Pose bỏ sót mục tiêu do nhòe hình

Như quan sát tại **Hình 3.19**, trong điều kiện lóa sáng nặng kết hợp với khoảng cách xa, các đường nét cấu trúc viền và ký tự bên trong biển số bị triệt tiêu hoàn toàn. Khối định vị không gian không có đủ dữ kiện để nhận diện đối tượng và trích xuất 4 điểm Keypoints, dẫn đến việc luồng xử lý bị ngắt ngay lập tức.



Hình 3.20. YOLO-Pose bỏ sót mục tiêu do biển số bị che khuất

Tương tự tại **Hình 3.20**, một kịch bản bỏ sót khi vùng biển số bị che khuất. Do người dùng không di chuyển vào đúng vùng quy định nên dẫn tới biển số bị khuất. Sự thất bại

trong việc kích hoạt hộp bao dự đoán lúc này là một giới hạn khách quan về mặt vật lý khi vận hành hệ thống, không phải là lỗi thuật toán mạng nơ-ron.

- **Lỗi nhận diện sai ký tự từ khối YOLO-OCR:**

- **Nhiều do vật cản:** đinh ốc cố định biển số nằm sát hoặc đè lên nét chữ; ký tự bị vật cản che khuất một phần; ... đều gây ảnh hưởng tới quá trình nhận diện.



Hình 3.21. Trường hợp hệ thống nhận diện bỏ sót

(Biển số dự đoán: 33M9629 – Biển số đúng: 33M69629)

- **Nhiều do môi trường:** ở các môi trường điều kiện không lý tưởng: góc camera xa/ chéo, camera độ phân giải không cao, ánh sáng quá chói hoặc quá tối, ... dẫn tới các hiện tượng ảnh bị mờ, nhiễu, ... và điều đó ảnh hưởng trực tiếp tới khả năng nhận diện của mô hình. Đối mặt với những ca lỗi này là minh chứng cho thấy hệ thống vẫn cần sự hỗ trợ trực tiếp từ con người hoặc sử dụng biện pháp hậu xử lý (chuẩn hóa ký tự).



Hình 3.22. Trường hợp hệ thống nhận diện sai

(Biển số dự đoán: 3024003 – Biển số đúng: 30Z4003)

Những trường hợp này xác định ranh giới vật lý của mô hình và là cơ sở để đề xuất các biện pháp hậu xử lý bằng logic nghiệp vụ trong tương lai.

3.5. Nghiên cứu thực nghiệm so sánh với Kiến trúc lai (YOLO11 + Thư viện PaddleOCR)

3.5.1. Mục tiêu và phương pháp kiểm thử

- **Mục tiêu:** Kiểm chứng thực nghiệm giả thuyết đã đề xuất tại Chương 1 (Mục 1.1.2) về tính ưu việt của Hệ thống nhận dạng đồng nhất (chỉ sử dụng họ mô hình YOLO11) so với Kiến trúc lai (kết hợp YOLO và một thư viện OCR độc lập mã nguồn mở).
- **Phương pháp:** Đề án thiết lập một luồng xử lý đối chứng (Baseline pipeline). Cụ thể, sau khi ảnh biển số được cắt và nắn phẳng bằng YOLO11-Pose kết hợp thuật toán Perspective Transform, thay vì đưa vào mô hình YOLO11-OCR, tensor ảnh sẽ được chuyển sang thư viện PaddleOCR để nhận diện chuỗi ký tự. Cả hai hệ thống (YOLO11 đồng nhất và YOLO11 + PaddleOCR) được thực thi độc lập trên cùng một tập dữ liệu kiểm thử thực tế.

3.5.2. Kết quả so sánh định lượng

Sau khi trích xuất và tổng hợp nhật ký thực thi (logs) từ quá trình chạy đối chứng trên toàn bộ tập dữ liệu, kết quả so sánh chi tiết được thể hiện qua bảng sau:

Bảng 3.2. Bảng đối chiếu hiệu năng thực nghiệm giữa hệ thống lõi họ YOLO11 và Kiến trúc lai (YOLO11 + PaddleOCR) trên CPU

Tiêu chí đánh giá	Pipeline YOLO11	YOLO11 + paddleOCR
Số lượng ảnh nhận diện đúng	6439	5229
Tỉ lệ độ chính xác toàn chuỗi	~92.91%	~75.45%
Thời gian xử lý trung bình	~90.76 ms	~151.6 ms

3.5.3. Biện luận kỹ thuật và Kết luận

Từ dữ liệu thực nghiệm tại Bảng 3.2, đồ án rút ra hai luận điểm kỹ thuật then chốt để bảo vệ kiến trúc hệ thống đã chọn:

- **Về mặt độ chính xác toàn chuỗi (Full-Sequence Accuracy):** Kiến trúc lai kết hợp PaddleOCR ghi nhận mức sụt giảm nghiêm trọng, chỉ đạt ~75.45% so với 92.91% của hệ thống YOLO11. Nguyên nhân cốt lõi là do PaddleOCR được tối ưu hóa cho bài toán nhận diện văn bản trên tài liệu dạng phẳng (Document OCR). Qua quá trình quan sát log nhận diện, khi đối mặt với dữ liệu biển số xe thường xuyên bị lóa sáng, mờ, dính bùn đất hay có đỉnh ốc che khuất nét chữ, PaddleOCR gặp khó khăn lớn, thường xuyên nhận diện sai các ký tự đồng dạng (như đọc "24V" thành "2LV") hoặc mất dấu hoàn toàn dẫn đến kết quả trả về None. Ngược lại, mô hình YOLO11-OCR được huấn luyện trực tiếp trên tập dữ liệu biển số đặc thù nên mạng nơ-ron có khả năng khoanh vùng đặc trưng hình thái ký tự và chống nhiễu vượt trội hơn hẳn.
- **Về mặt tốc độ suy luận (Inference Latency):** Hệ thống đồng nhất tối ưu thời gian xử lý nhanh gấp hơn 1.6 lần (~90.76 ms so với 151.6 ms). Việc luân chuyển dữ liệu ảnh sau khi nắn phẳng từ môi trường của YOLO sang môi trường thực thi của thư viện PaddleOCR tạo ra chi phí trễ (Overhead) và nghẽn luồng xử lý nghiêm trọng. Trong khi đó, việc chuẩn hóa toàn bộ Pipeline trên một họ cấu trúc duy nhất (Ultralytics) cho phép bộ biên dịch Intel OpenVINO tối ưu hóa triệt để, thực hiện gộp lớp (Layer Fusion) liên hoàn và tận dụng tối đa tài nguyên Cache của vi xử lý CPU x86.

Kết luận: Thử nghiệm đối chứng đã chứng minh một cách thực tiễn rằng: Giải pháp thiết kế hệ thống nhận diện đồng nhất bằng cấu trúc YOLO11 giải quyết trọn vẹn cả hai mục tiêu cốt lõi của đồ án. Nó không chỉ mang lại độ chính xác cao nhất mà còn duy trì được tốc độ phản hồi lý tưởng (thời gian thực) ngay cả khi triển khai trên phần cứng hạn chế (CPU).

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Sau thời gian nghiên cứu và thực hiện đề tài " Nghiên cứu và phát triển API nhận diện biển số xe tự động ứng dụng Deep Learning", đề án đã hoàn thành các mục tiêu đề ra ban đầu. Những kết quả chính mà hệ thống đã đạt được bao gồm:

- **Về mặt nghiên cứu thuật toán:** Đã làm chủ và xây dựng thành công luồng xử lý đồng nhất dựa trên kiến trúc YOLO để giải quyết trọn vẹn ba bài toán: Phân loại phương tiện, Phát hiện biển số và Nhận dạng ký tự (OCR). Đặc biệt, việc ứng dụng biến thể YOLO-Pose để trích xuất tọa độ 4 góc (Keypoints) đã giúp hệ thống khắc phục triệt để hiện tượng biến dạng phối cảnh thông qua phép biến đổi Perspective Transform.
- **Về mặt tối ưu hóa hiệu năng:** Bằng việc ứng dụng framework Intel OpenVINO và kỹ thuật lập trình bất đồng bộ (Asynchronous) trong FastAPI, đề án đã chứng minh khả năng vận hành thời gian thực trên các thiết bị chỉ trang bị vi xử lý trung tâm (CPU). Tốc độ phản hồi trung bình đạt mức ~90ms cho một luồng xử lý toàn diện.
- **Về mặt thực nghiệm:** Hệ thống đã được kiểm chứng trên tập dữ liệu thực tế gồm 6.930 ảnh với nhiều kích bản phức tạp, đạt tỷ lệ chính xác toàn chuỗi là 92,91%. Kết quả này khẳng định tính tin cậy và khả năng ứng dụng cao của mô hình trong thực tiễn quản lý bãi xe thông minh.
- **Về mặt triển khai:** Toàn bộ hệ thống đã được đóng gói thành công dưới dạng Container bằng công nghệ Docker, đảm bảo tính di động, ổn định và sẵn sàng tích hợp linh hoạt vào các hạ tầng phần mềm khác.

2. Những hạn chế của đề án

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế cần được khắc phục:

- Mô-đun phân loại thương hiệu phương tiện (Luồng 1) hiện tại mới chỉ đóng vai trò thử nghiệm tính khả thi với quy mô dữ liệu còn hạn chế (~400 ảnh), dẫn đến độ ổn định chưa cao ở các dòng xe hiếm gặp.
- Độ chính xác của thuật toán OCR vẫn bị ảnh hưởng trong các trường hợp cực đoan như biển số bị che khuất quá nhiều bởi vật cản (đỉnh ốc, bùn đất) hoặc hình ảnh bị nhiễu nặng do điều kiện ánh sáng quá tối.

3. Hướng phát triển trong tương lai

Để hoàn thiện hệ thống và đưa vào ứng dụng rộng rãi hơn, đề án định hướng các bước phát triển tiếp theo như sau:

- **Nghiên cứu và chuyển đổi kiến trúc hệ thống:** Trong tương lai, khi hệ thống yêu cầu triển khai trên các thiết bị Edge AI có cấu hình siêu thấp, hệ thống sẽ được nghiên cứu để nâng cấp lên các kiến trúc nhẹ hơn như YOLO26 nhằm tối ưu hóa triệt để dung lượng phần cứng.
- **Mở rộng tập dữ liệu:** Tiếp tục thu thập và huấn luyện bổ sung các mẫu ảnh trong điều kiện thời tiết xấu (mưa, sương mù) và các góc camera đặc thù để nâng cao khả năng tổng quát hóa của mô hình.
- **Triển khai trên thiết bị nhúng:** Tối ưu hóa sâu hơn nữa để đưa hệ thống API vận hành trực tiếp trên các thiết bị biên (Edge Devices) như Raspberry Pi hoặc NVIDIA Jetson, hướng tới giải pháp camera AI tích hợp hoàn chỉnh.
- **Hậu xử lý thông minh:** Xây dựng thêm bộ lọc logic dựa trên quy chuẩn biển số xe Việt Nam (Regex) để tự động sửa lỗi cho các cặp ký tự dễ nhầm lẫn (như 8-B, 0-D), từ đó nâng tỷ lệ chính xác toàn chuỗi lên mức tối đa.
- **Tích hợp thuật toán Tracking:** Nghiên cứu và triển khai thêm các module theo dõi đối tượng (như ByteTrack hoặc DeepSORT) để hệ thống có khả năng nhận diện biển số xe liên tục trên luồng video thay vì chỉ xử lý ảnh tĩnh.

TÀI LIỆU THAM KHẢO

Dựa trên danh sách các tài liệu bạn đã cung cấp trong báo cáo và nội dung kỹ thuật từ các file .py (sử dụng YOLO, OpenVINO, FastAPI), tôi đã chuẩn hóa lại danh mục Tài liệu tham khảo theo **chuẩn IEEE**.

Bạn hãy copy đoạn này vào mục **TÀI LIỆU THAM KHẢO** trong báo cáo ĐATN của bạn:

TÀI LIỆU THAM KHẢO

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobb's J. Softw. Tools*, vol. 25, no. 11, pp. 120–123, 2000.
- [3] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [4] Python Software Foundation, "Python Language Reference, version 3.11," 2023. [Online]. Available: <https://www.python.org>. [Accessed: May 25, 2026].
- [5] G. Jocher et al., "Ultralytics YOLO," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed: May 25, 2026].
- [6] Intel Corporation, "Intel® Distribution of OpenVINO™ Toolkit Documentation," 2024. [Online]. Available: <https://docs.openvino.ai>. [Accessed: May 25, 2026].
- [7] S. Ramírez, "FastAPI Web Framework Documentation," 2020. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: May 25, 2026].
- [8] Encode OSS, "Uvicorn: The lightning-fast ASGI server," 2024. [Online]. Available: <https://www.uvicorn.org/>. [Accessed: May 25, 2026].
- [9] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com>. [Accessed: May 25, 2026].
- [10] Roboflow, "Roboflow: Platform for Computer Vision," 2024. [Online]. Available: <https://roboflow.com>. [Accessed: May 25, 2026].
- [11] Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, 2020. [Online]. Available: <https://arxiv.org/abs/2009.09941>. [Accessed: May 25, 2026].